國家實驗研究院
NATIONAL INSTITUTES OF APPLIED RESEARCH

深度學習之分散式訓練實作

方育斌 副研究員
高效能計算與雲端計算組
ybfang@narlabs.org.tw

教材下載：https://github.com/robinfang7/ddp_course

Outline

1. 國網中心的超級電腦
    1. Nano5 (Nvidia H100/H200 GPU cluster)
    2. Taiwania 2(Nvidia Tesla V100 GPU cluster)
    3. iService計算資源服務網
2. HPC Cluster Software
    1. SLURM任務調度
    2. Lmod環境管理
    3. Singularity container虛擬環境管理
        1) Customized image打包客製化環境
3. 案例實作
    1. Horovod
    2. Pytorch DDP
    3. DeepSpeed

Nano5 (晶創主機)

晶創25(英文名稱：Nano 5)為新一代加速器叢集式超級電腦，計有21台NVIDIA H100以及16台NVIDIA H200AI伺服器，共有296片NVIDIA Hopper計算加速器，為台灣發展AI及高效能運算的關鍵基礎設施。晶創主機也承襲「晶創計畫」的願景，助力半導體產業發展AI應用，也為半導體產業其他各種應用及研發，提供強大且高速的計算環境。晶創25主機採用NVIDIA Hopper計算加速器，於2025年6月TOP500首次排行118名，算力(Rmax)達13.06PetaFLOPS，可大幅提升人工智慧模型訓練與科學模擬的運算效率。此主機適用於產業界、政府機構、學術單位以及研究機構，為各類用戶提供高效能的運算支援，助力AI全面推動。

Taiwania 2 台灣杉二號 / TWCC台灣AI雲

「台灣杉二號」(Taiwania 2)為AI超級電腦主機，共運用2,016個NVIDIA Tesla V100 32GB GPU，以9 PFLOPS（每秒執行9千兆次浮點運算)的優異效能，在2018年底公布的全球500大高速計算主機（TOP500）中，排名第20名，僅次於美國、中國、瑞士、日本、德國、韓國、義大利、法國；能源效率（Green500）排名第10名，僅次於日本、美國、西班牙、中國，雙雙締造台灣超級電腦入榜有史以來最好的成績，也寫下我國科技國力發展的重要里程碑。

「台灣杉二號」(Taiwania 2)超級電腦由252個節點組成，每個節點包含2顆CPU及8顆最先進GPU，其主機架構設計與國際趨勢同步。在科學家運用大數據進行深度學習時，可表現出更優質的性能；此外，在節能方面，「台灣杉二號」的能源效率達11.285 GF/W，計算量在9 PFLOPS時，用電798 KW，亦為台灣史上最節能的高速計算主機。

規格

國家實驗研究院 NATIONAL INSTITUTES OF APPLIED RESEARCH

	晶創主機	台灣杉二號
建置時間	2025	2018
總算力	13.06 PFLOPS	9 PFLOPS
節點數	21台H100節點+16台H200節點	252
檔案系統總容量	9.3 PB	10 PB
節點CPU	2顆Intel Xeon(R) Platinum 8480+(8480CL)	Intel Xeon Gold CPU x 2
節點GPU	8片NVIDIA H100 GPU 8片NVIDIA H200 GPU	Nvidia Tesla V100 w/32GB x 8
節點記憶體容量	2TB	768 GB
網路頻寬	計算伺服器間以8個Infiniband NDR 400Gbs網路埠連接 以2個 Infiniband NDR 200Gbs網路埠連接至儲存系統	InfiniBand EDR100 100Gbps

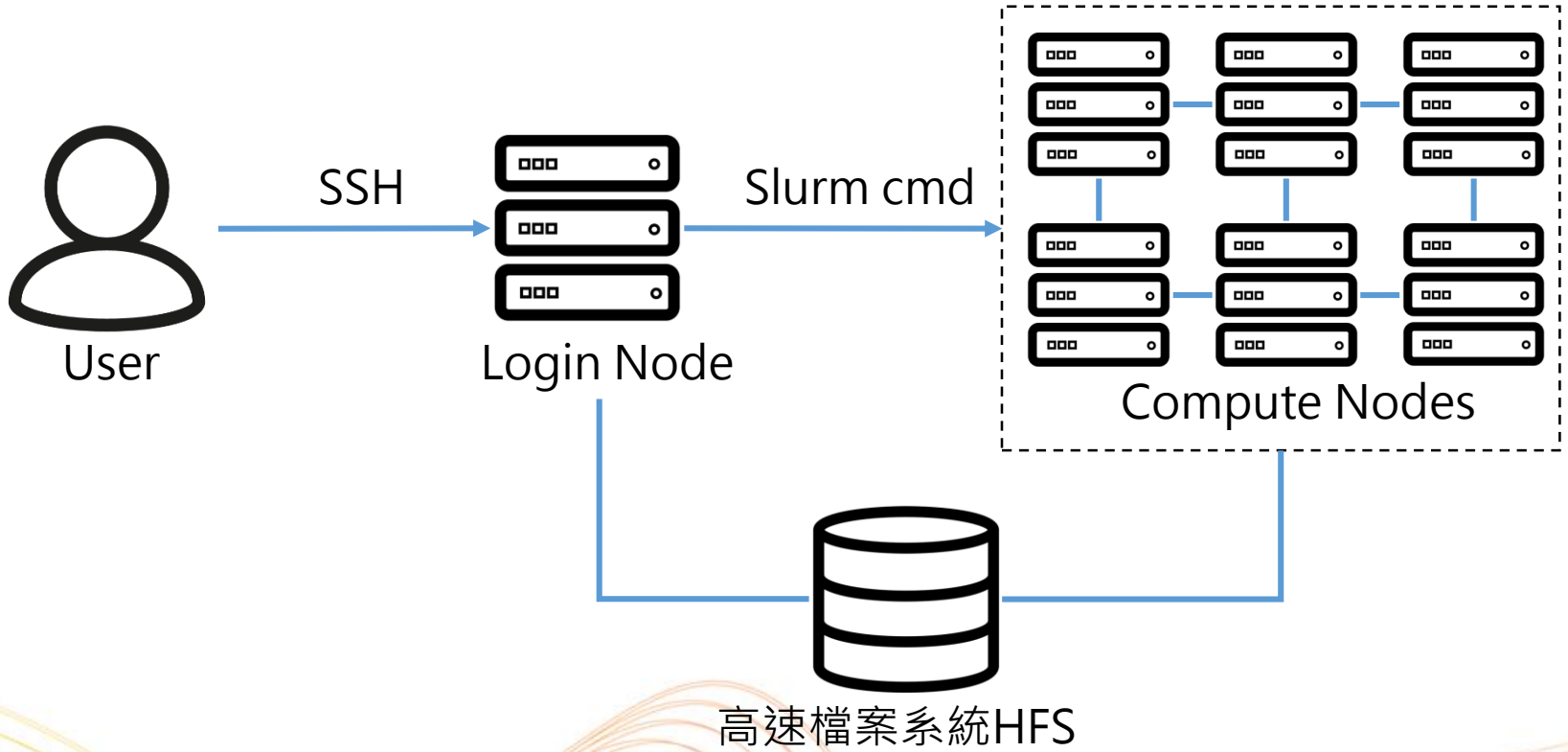
GPU Spec

	H100 SXM5 80 GB	H200 SXM 141 GB	Tesla V100 SXM2 32 GB
Release Date	Oct. 2022	Nov. 2024	Mar. 2018
Process Size	5nm	5nm	12nm
Transistors	80,000 million	80,000 million	21,100 million
Memory Size	80 GB	141 GB	32 GB
Memory Type	HBM3	HBM3e	HBM2
Bandwidth	3.36 TB/s	4.89 TB/s	898.0 GB/s
FP16 (half)	267.6 TFLOPS	267.6 TFLOPS	31.33 TFLOPS
FP32 (float)	66.91 TFLOPS	66.91 TFLOPS	15.67 TFLOPS
FP64 (double)	33.45 TFLOPS	33.45 TFLOPS	7.834 TFLOPS
TDP	700 W	700 W	250 W
GPU小時費率	20/50/50/120	30/60/60/150	86(企業與個人計畫)

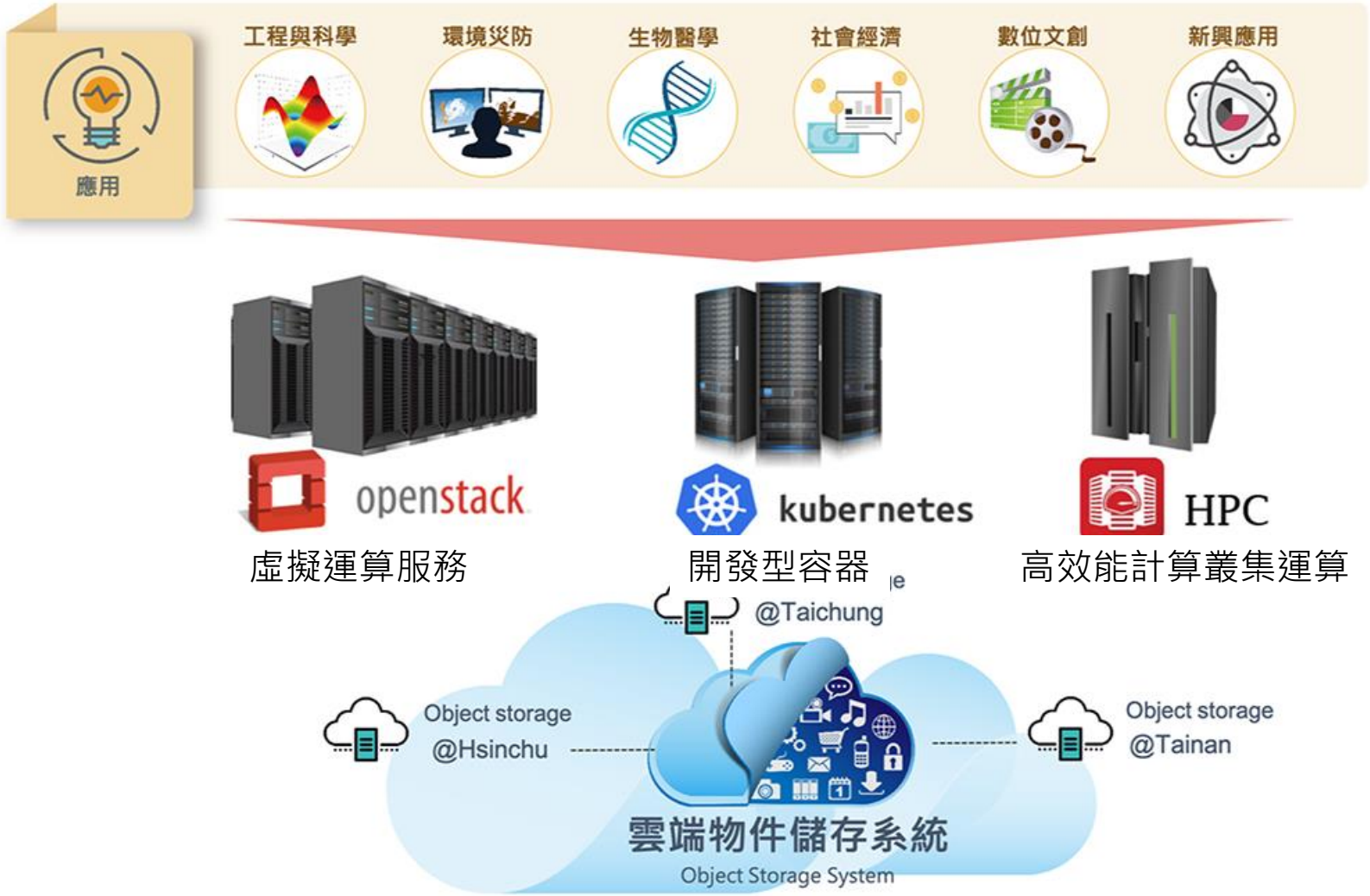
四種GPU小時費率為國科會計畫、學術計畫、政府與法人計畫、企業與個人計畫

https://www.techpowerup.com/gpu-specs/tesla-v100-sxm2-32-gb.c3185
https://www.techpowerup.com/gpu-specs/h100-sxm5-80-gb.c3900
https://www.techpowerup.com/gpu-specs/h200-sxm-141-gb.c4255

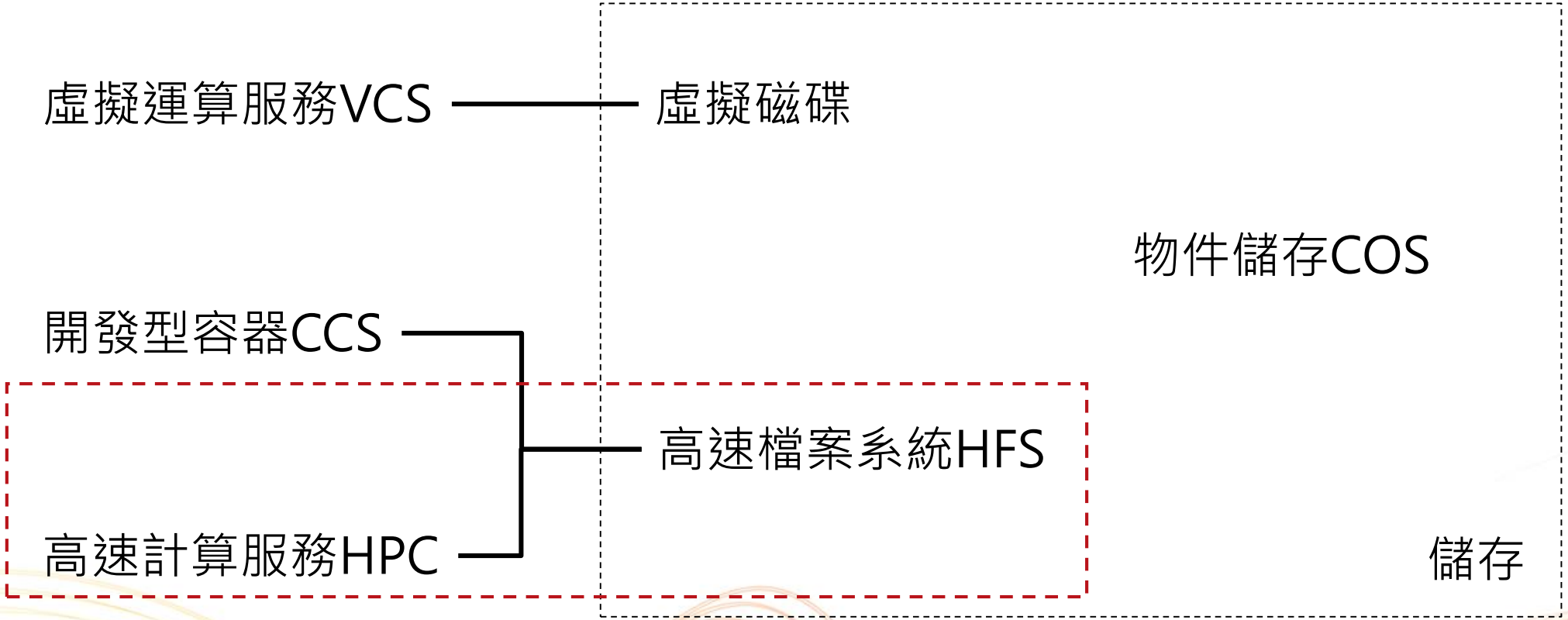
HPC cluster



TWCC軟體架構

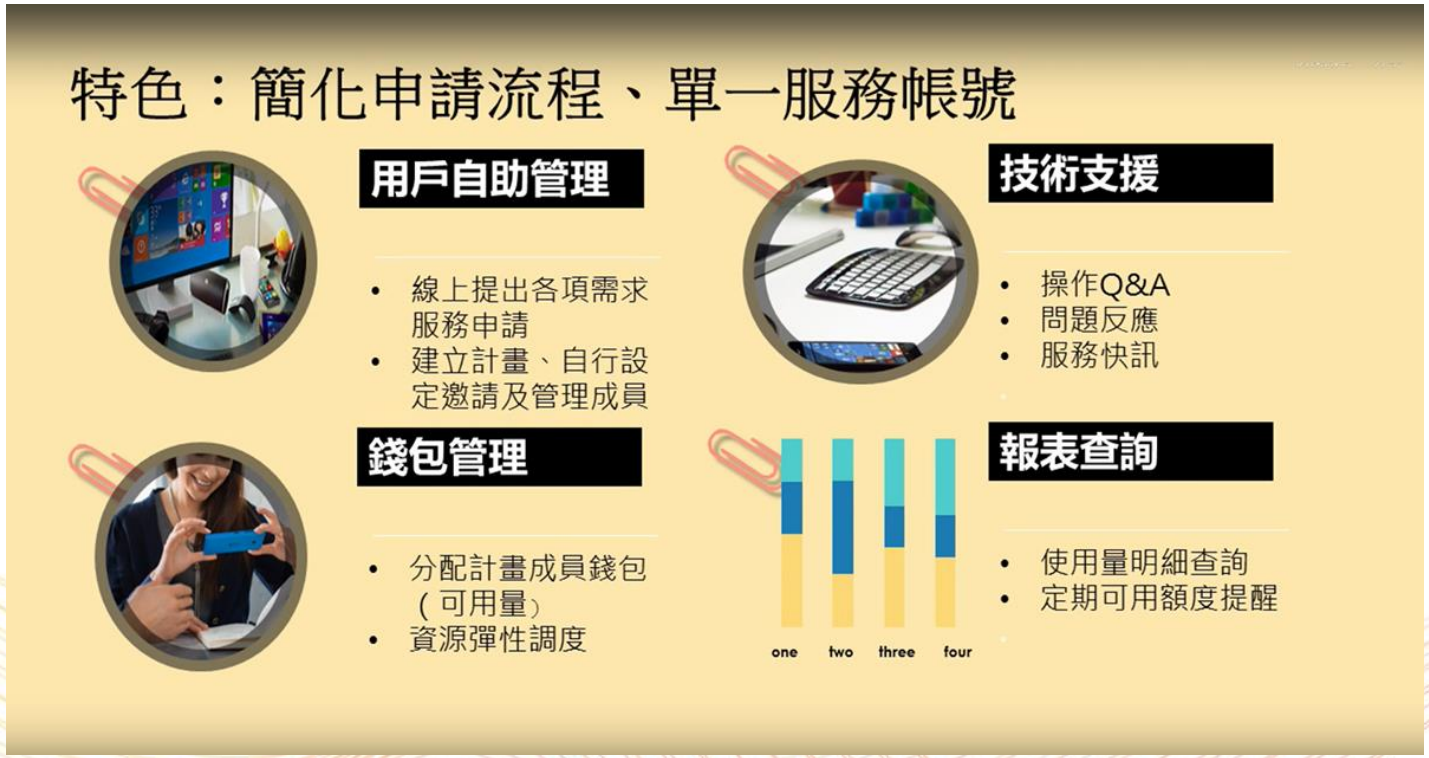

應用
工程與科學
環境災防
生物醫學
社會經濟
數位文創
新興應用
openstack
kubernetes
HPC
虛擬運算服務
開發型容器
高效能計算叢集運算
@Taichung
Object storage
@Hsinchu
Object storage
@Tainan
雲端物件儲存系統
Object Storage System

TWCC服務說明



iService計畫資源服務網

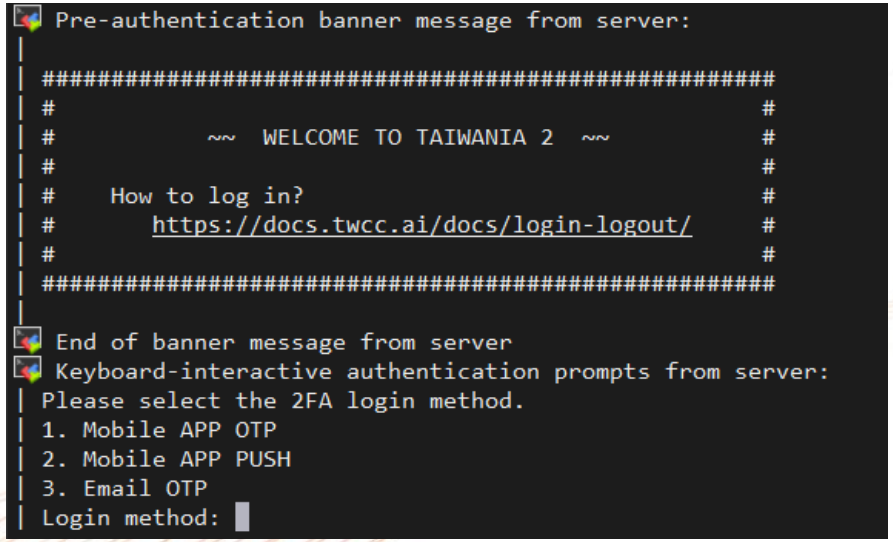
- https://iservice.nchc.org.tw/
- 提供國網中心各服務平台的帳戶申請及管理，包含計畫成員管理、預算和經費管理、使用資源報表查詢、平台操作技術支援等功能，其中計算資源申請服務，如創進一號、臺灣AI雲(TWCC)等，透過iService計算資源服務網提供服務。



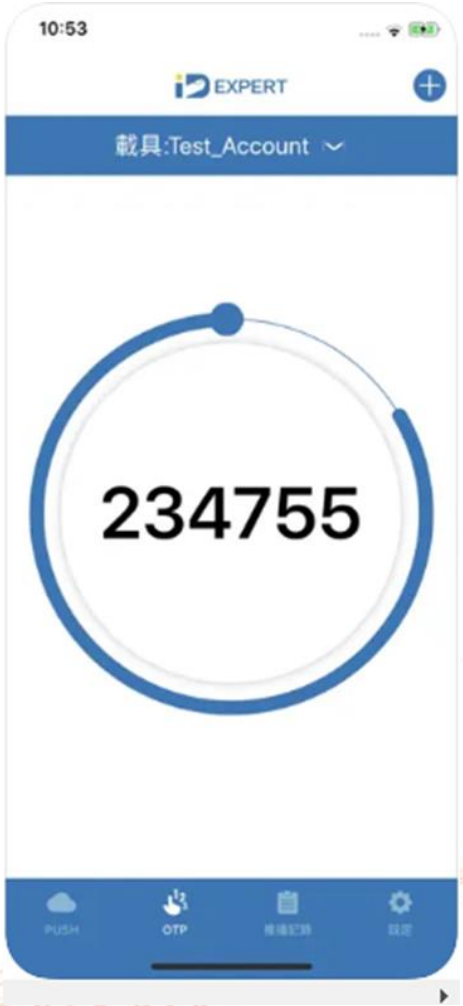
帳號有兩種

- iService會員帳號
  - 登入iService網站、TWCC網站
  - 帳號的格式是電子信箱

- 主機帳號
  - 登入HPC主機，創進一號、 晶創主機、Taiwania 2



主機帳號與OTP設定



- https://iservice.nchc.org.tw/module_page.php?module=nchc_service#nchc_service/nchc_service.php?action=nchc_motp_unix_account_edit_v3
- https://iservice.nchc.org.tw/nchc_service/nchc_service_qa_single.php?qa_code=774

高速檔案系統 HFS Hyper File System

	晶創主機	台灣杉二號
免費額度	/home 100GB /work 100GB (國科會計畫1536GB)	/home 100 GB /work 100 GB (國科會計畫1500GB)
最高容量	/home 1024 GB (1TB) /work 10240 GB (10TB)	/home 10240 GB (10TB) /work 20480 GB (20TB)
Hostname	nano5.nchc.org.tw	xdata1.twcc.ai
Port	port 2222	port 22

收費方法：每日計費，每天根據前一日的用量平均值進行扣款，以更實際地反應用戶的使用情況。
HFS儲存空間：(*Home、Work與Project 費率相同) (單位：元)

計畫類型	每日1 GB
國科會計畫	0.0069
學術計畫	0.035
政府與法人計畫	0.07
企業與個人計畫	0.14

https://man.twcc.ai/iPIMu_XGQcCNNTRyJMAPmg
https://man.twcc.ai/@twccdocs/doc-hfs-main-zh/%2F%40twccdocs%2Fhfs-overview-zh
https://iservice.nchc.org.tw/nchc_service/nchc_service_qa_single.php?qa_code=802

設定與查詢HFS 容量

國家實驗研究院 NATIONAL INSTITUTES OF APPLIED RESEARCH

計劃建立者和管理者有權限設定HFS容量

查詢HFS容量 `/usr/lpp/mmfs/bin/mmlsquota --block-size auto fs01 fs02`

```
[u8880716@un-ln01 ~]$ /usr/lpp/mmfs/bin/mmlsquota --block-size auto fs01 fs02
                         Block Limits                                    |     File Limits
Filesystem type         blocks      quota      limit   in_doubt    grace |    files    quota    limit in_doubt    grace  Remarks
fs01       USR           1.79T        20T       20.1T          0     none |  1545320        0        0        0     none NCHC_AIcls.twcc.ai
fs02       USR          2.621T        10T       10.1T          0     none |  1597391        0        0        0     none NCHC_AIcls.twcc.ai
```

/work
/home

hfsquota

```
[u8880716@cbi-lgn01 ~]$ hfsquota
PATH              USED              HARD LIMIT          USAGE %  STATUS
home:/u8880716    734600540160 B    107374182400000 B   0        ACTIVE

PATH              USED              HARD LIMIT          USAGE %  STATUS
work:/u8880716    571715866624 B    107374182400000 B   0        ACTIVE
```

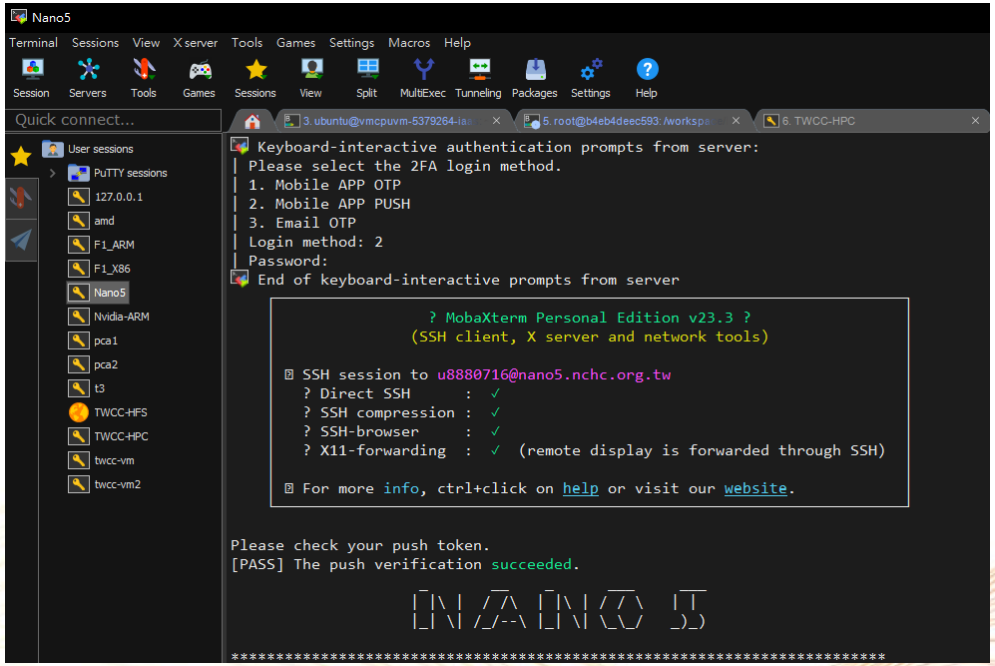
734600540160 bytes /1024/1024/1024=684GB

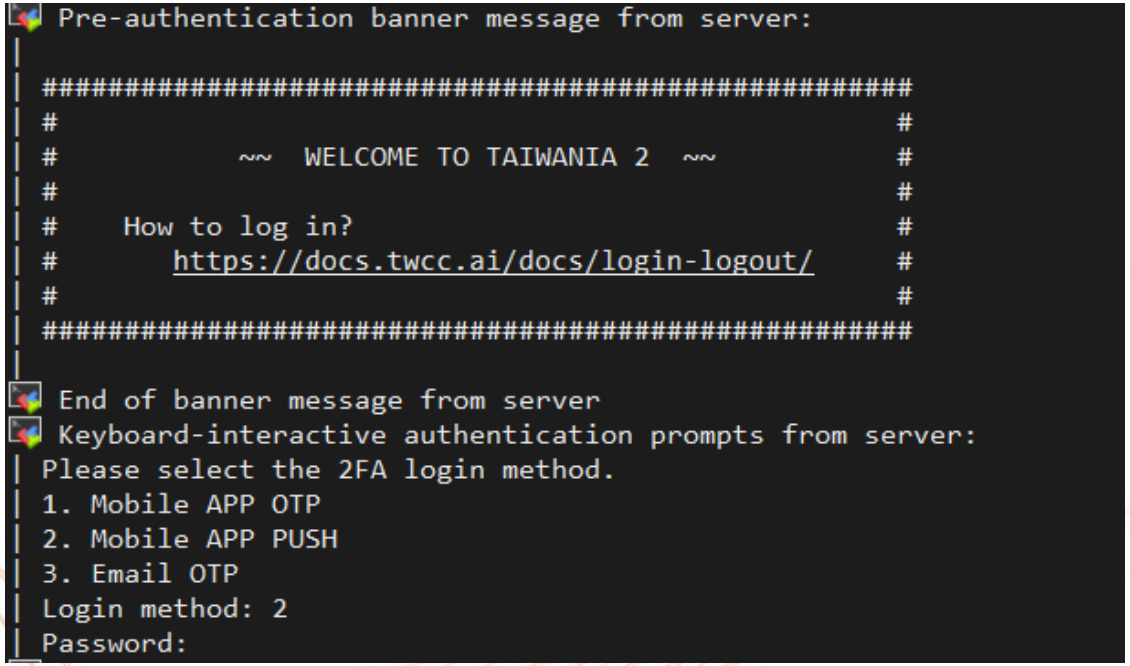
https://man.twcc.ai/@TWCC-III-manual/HyOgKliuu

登入節點

- SSH Tools
  - MobaXterm, Putty for Windows
  - Terminal for Mac and Linux
- 雙因子(2FA) 驗證

	晶創主機	台灣杉二號
Host name	nano5.nchc.org.tw	ln01.twcc.ai
Port	22	22



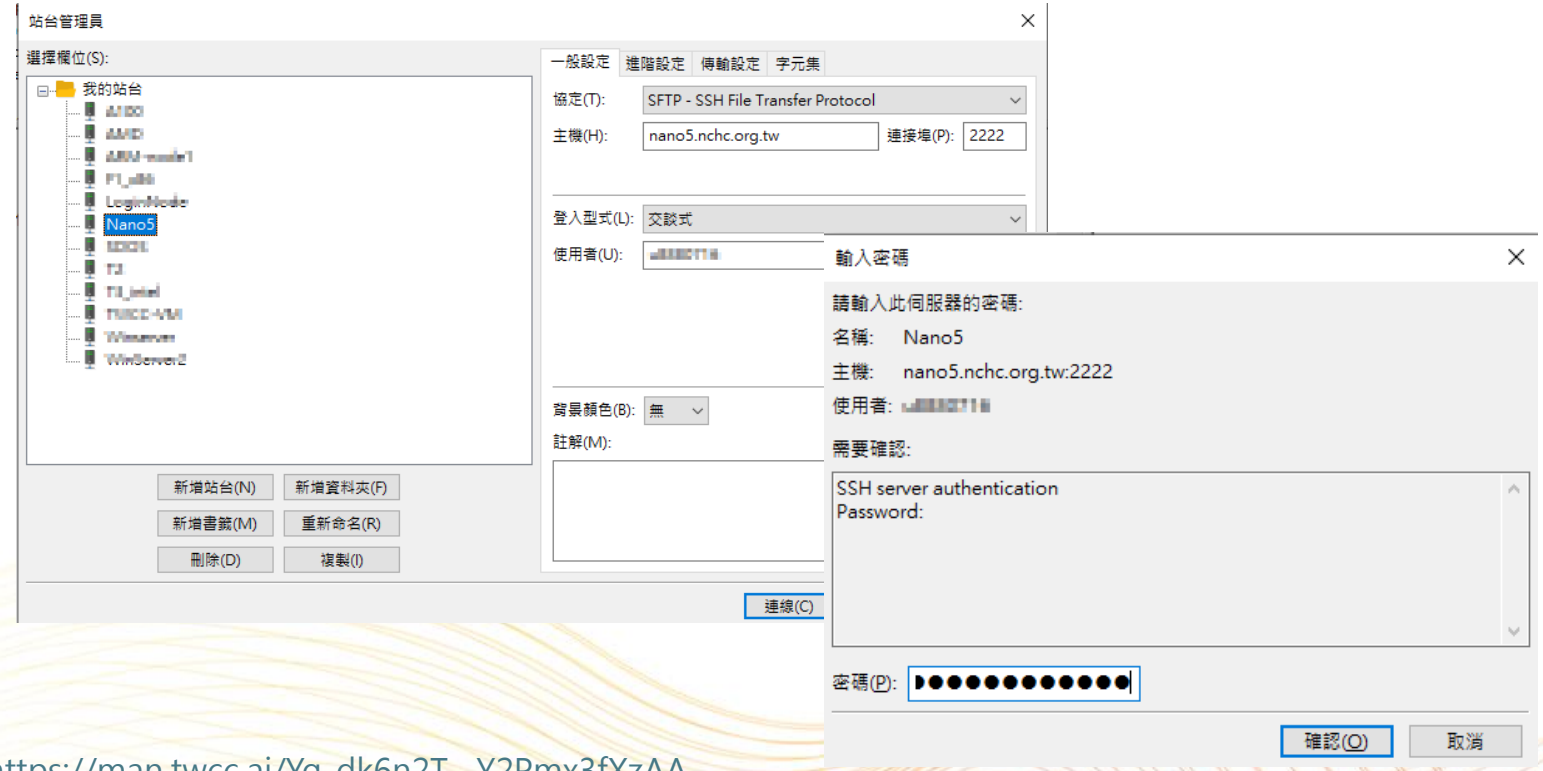


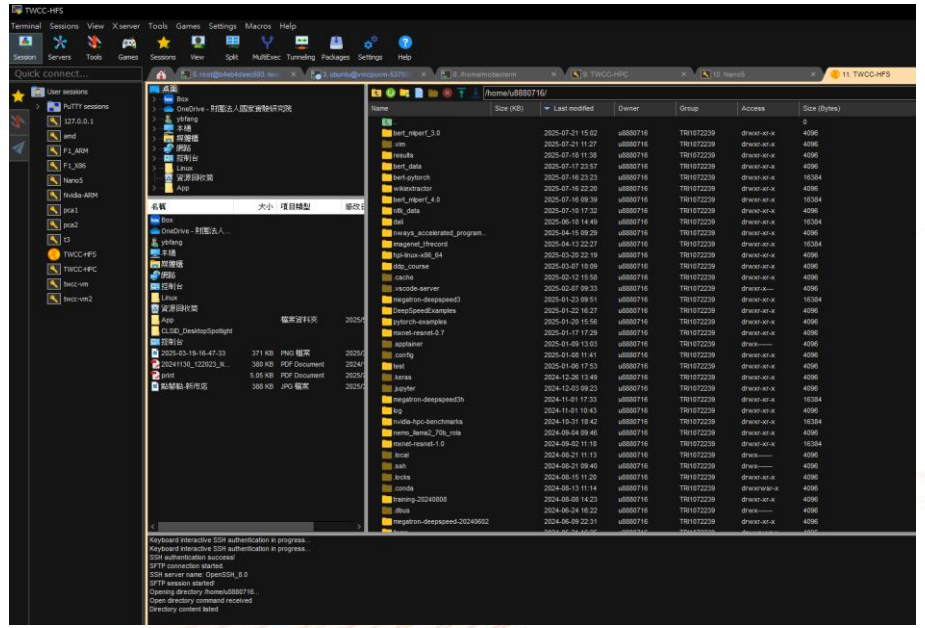
https://man.twcc.ai/KluSqJFJQi6SrNJQRKPJdA
https://man.twcc.ai/@twccdocs/doc-twnia2-main-zh/https%3A%2F%2Fman.twcc.ai%2F%40twccdocs%2Fguide-twnia2-login-and-logout-zh

資料傳輸節點

- SFTP Tools
  - Filezilla, WinSCP, MobaXterm
  - sftp command
- 雙因子(2FA) 驗證

	晶創主機	台灣杉二號
Hostname	nano5.nchc.org.tw	xdata1.twcc.ai
Port	port 2222	port 22

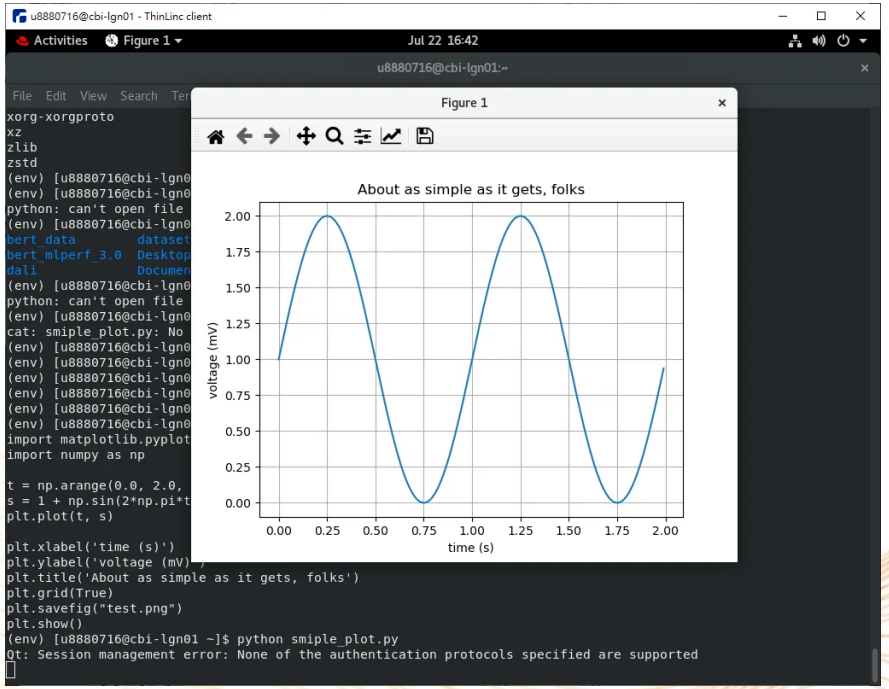


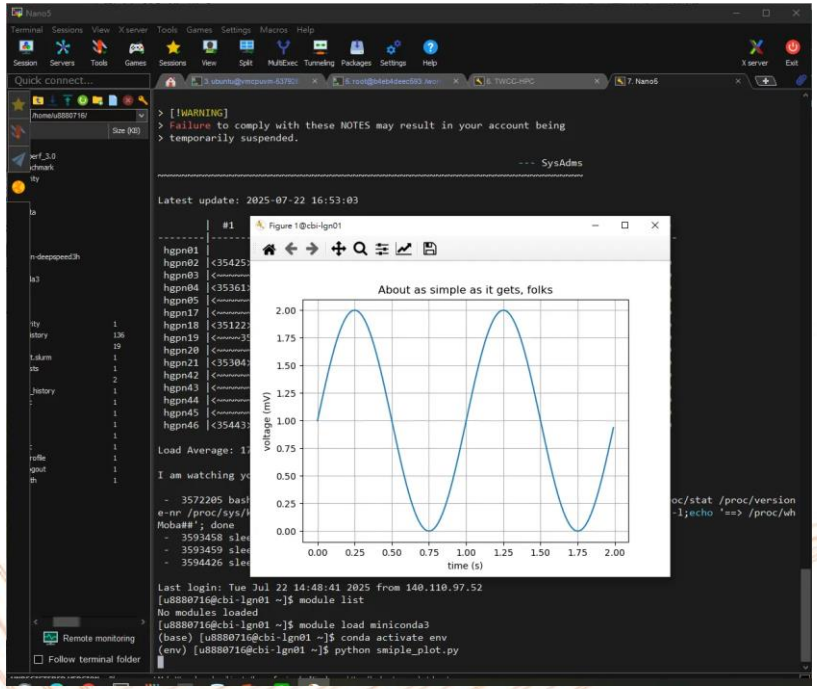


https://man.twcc.ai/Yg_dk6n2T_-Y2Pmx3fXzAA
https://man.twcc.ai/@twccdocs/doc-hfs-main-zh/%2F%40twccdocs%2Fhfs-overview-zh

登入晶創主機的互動式繪圖節點

- Tools
  - ThinLinc for Windows, Mac, and Linux
  - MobaXterm for Windows
- 雙因子(2FA) 驗證
- 節點名稱：naro5.nchc.org.tw



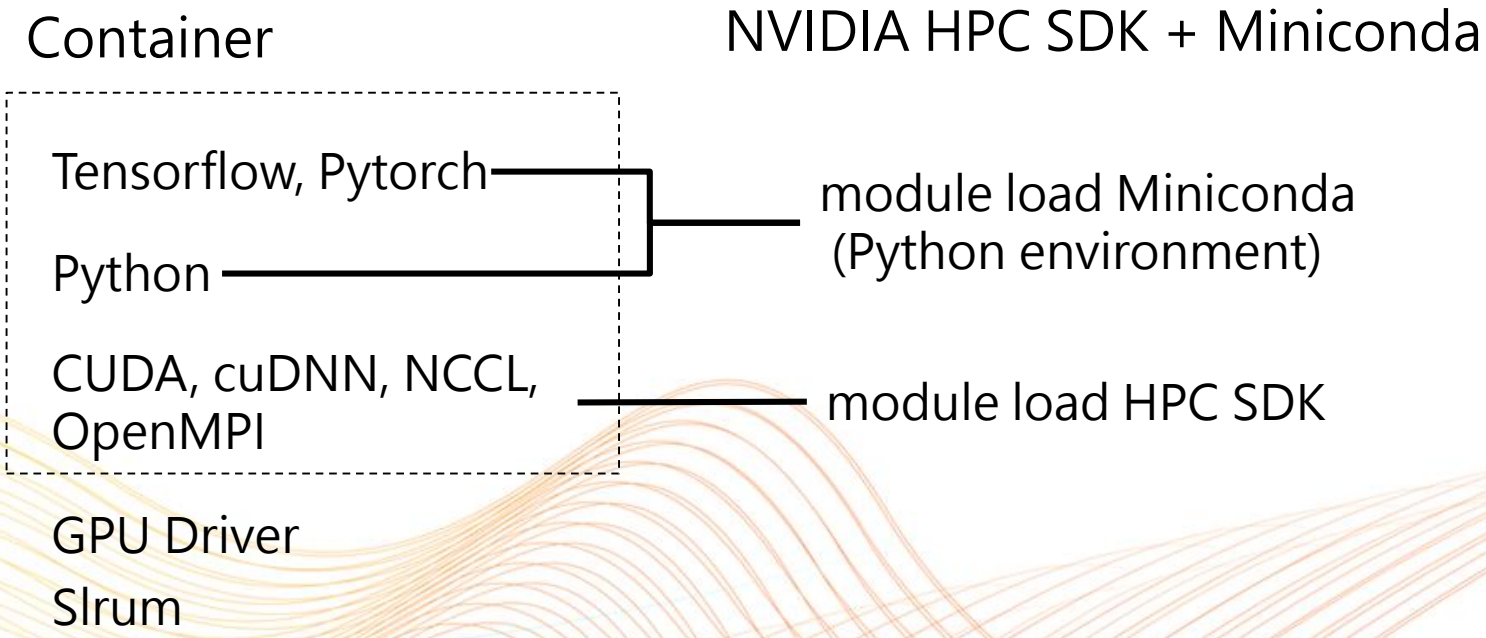


https://man.twcc.ai/B5MmH5TiS6qwCKYZnP7HBw

HPC Cluster Software

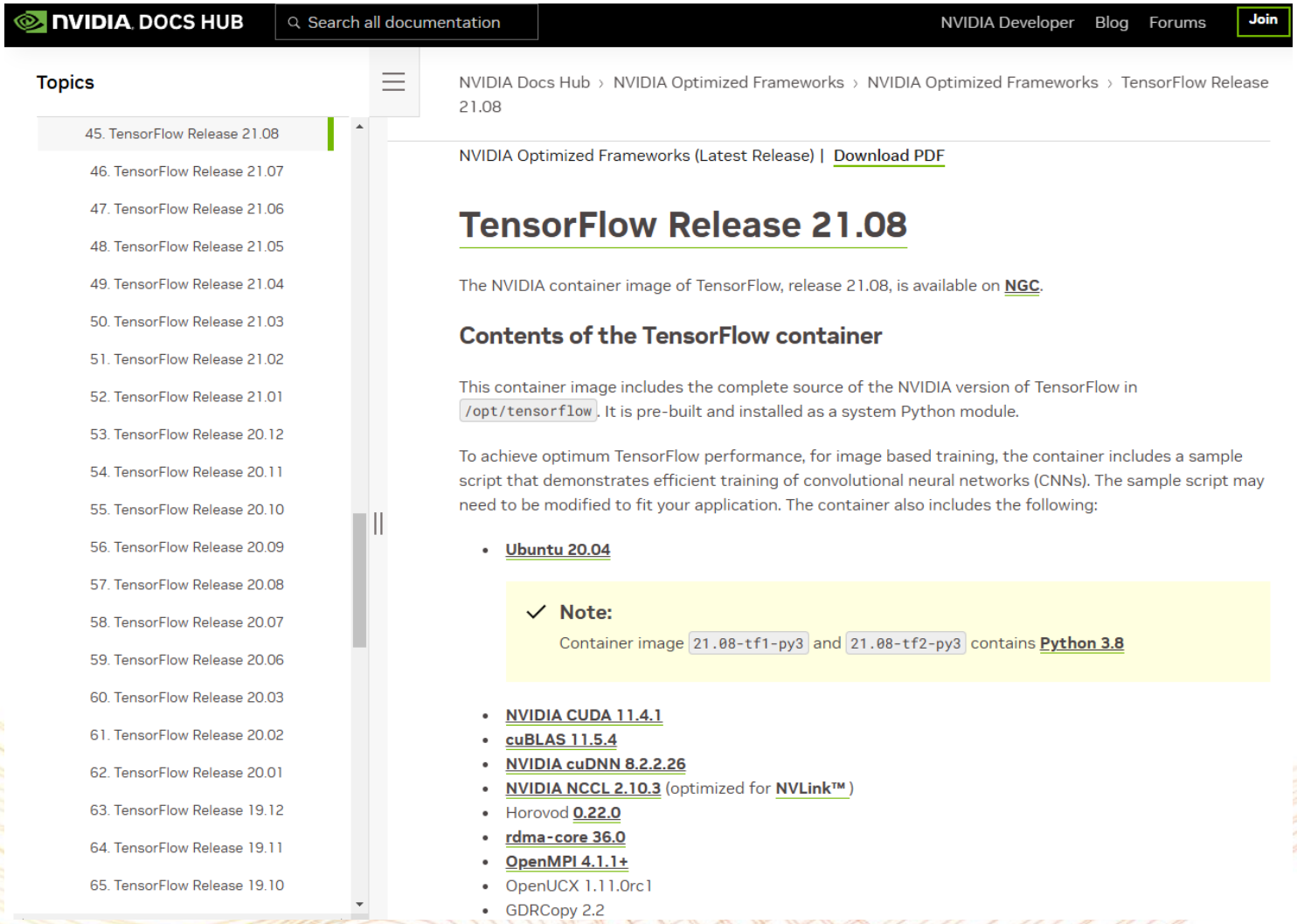
- Slurm任務調度工具
- Modules環境管理工具
- Singularity container虛擬環境管理工具

在HPC叢集系統，建立深度學習運算的軟體堆疊



Nvidia ngc container

https://docs.nvidia.com/deeplearning/frameworks/tensorflow-release-notes/rel_21-08.html



Slurm

Slurm 任務調度工具（極簡Linux資源管理工具，英文：Simple Linux Utility for Resource Management，取首字母，簡寫為SLURM），是一個用於 Linux 和 Unix 內核系統的自由開源的任務調度工具，被世界範圍內的超級計算機廣泛採用。

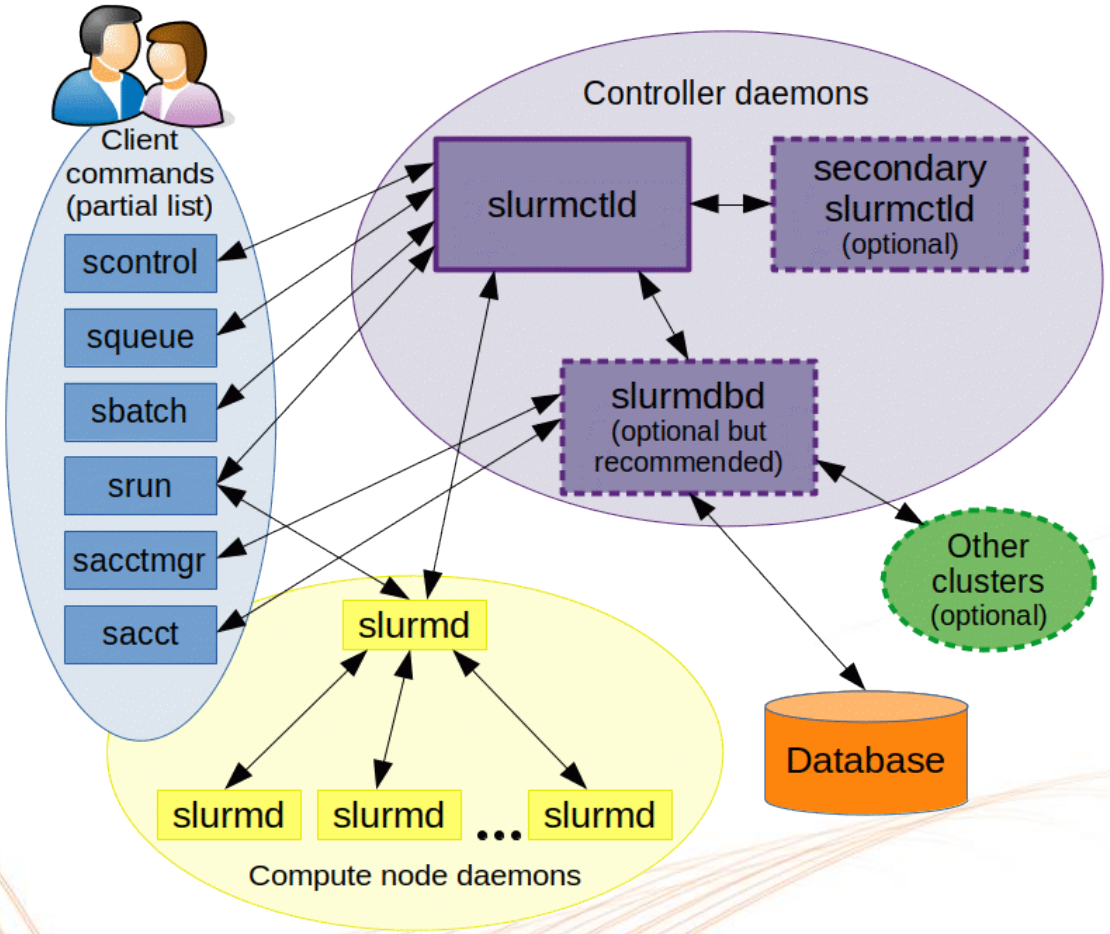
Slurm的功能。

- 資源分配
  Slurm 可以根據用戶提交的作業需求（例如節點數、CPU 核數、記憶體大小等）動態分配叢集中的計算資源，支援獨佔或非獨佔模式，保證資源得到最有效的利用。
- 作業排程與調度
  Slurm 根據優先順序、作業依賴性以及當前資源狀態，將待執行作業排入隊列並分配至合適的計算節點上。它支援多種排程策略（如先進先出、優先權調度、搶占式調度等），使得系統在面對大量作業請求時能保持高效率。
- 作業管理與監控

  用戶可以使用一系列命令（如 sbatch 提交批次作業、srun 執行互動作業、squeue 查詢作業狀態、scancel 取消作業等）來管理和監控作業進度，並透過記帳功能追蹤各作業的資源使用情況。

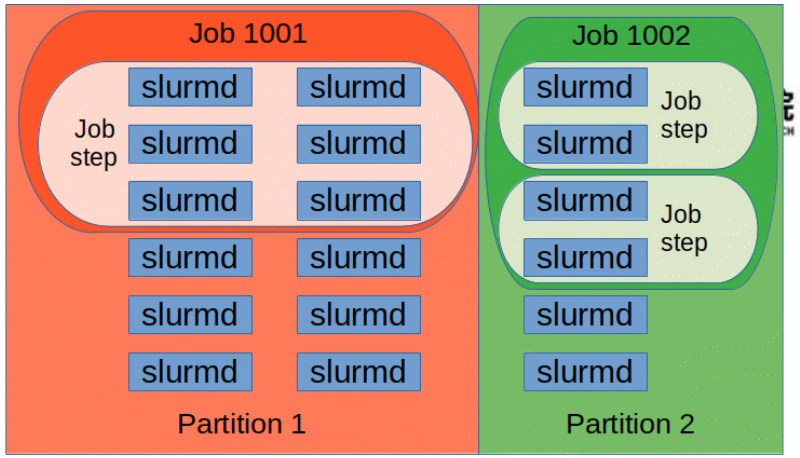
Slurm architecture

- slurmctld daemon：是 Slurm 的中央管理進程，負責監控集群資源和工作負載。
  - 資源監控: 追蹤集群中所有節點的可用資源，例如 CPU、記憶體等。
  - 作業排程與分配: 接收用戶提交的作業請求，根據資源需求和排程策略，將作業分配到可用的計算節點上。
  - 高可用性: 可以配置備份控制器 (backup slurmctld) 以應對主控制器故障，確保系統的穩定運行。
- slurmd daemon ：在集群中的每個計算節點上運行
  - 節點管理: 負責管理計算節點，監控節點的狀態，並向 slurmctld 回報。
  - 作業執行: 接收來自 slurmctld 的作業指令，在本地節點上啟動、執行和監控作業。
  - 狀態回報: 將節點和作業的狀態資訊回傳給 slurmctld。
- slurmdb daemon： Slurm 可以使用資料庫來儲存和管理集群的配置資訊、作業歷史記錄和帳戶資訊等。這有助於進行資源管理和追蹤。



https://slurm.schedmd.com/quickstart.html#arch

Slurm entities



Node

- 叢集中的伺服器或電腦。 這些是實際執行使用者作業的硬體資源。

Partition

- 分區將叢集中的節點按照用途、硬體配置或管理政策劃分為不同群組，類似於工作隊列。
- 用戶在提交作業時需要指定目標分區，每個分區可以設定不同的資源限制與排程政策。

Job

- 用戶提交的計算任務，記錄了資源需求、作業狀態、提交時間、執行時間、輸出檔案等資訊。
- 每個作業都有一個唯一的作業識別碼（Job ID），系統根據該識別碼追蹤作業進度。

Job steps

- 作業可以進一步細分為多個作業步驟，每個步驟通常對應一次 srun 命令的執行。
- 這允許同一個作業內部進行多階段或並行計算，並分別監控每個步驟的狀態與資源使用情形。

Tasks

- 一個job step可以包含一個或多個tasks，每個task代表一個要在計算節點上執行的工作。
- 在平行計算中，tasks通常用於執行同一個任務的多個實例，從而加速計算的進行。

Partition

國家實驗研究院 NATIONAL INSTITUTES OF APPLIED RESEARCH

晶創主機

佇列名稱	每個計劃最多可用GPU總數	每個Job最大執行時間	每個計畫同一時間		備註
			最多可執行job的數量	最多可進到主機排隊的Job數量	
dev	8	2 (小時)	2	2	H100
normal	16	48 (小時)	2	2	H100
normal2	16	48 (小時)	2	2	H200
4nodes	32	12 (小時)	2	2	H100

台灣杉二號

Queue 名稱	最長執行時間(小時)	高優先權	每位用戶最多 可提交計算工作數上限	適用計畫	每位用戶最多 可取用 GPU 數上限
gp1d	24		20	各式計畫	40
gp2d	48		20	各式計畫	40
gp4d	96		20	各式計畫	80
gtest	0.5		5	各式計畫	40
express	96	v	20	企業與個人計畫	256

https://man.twcc.ai/aPiCU8VXS7SZgFJBOoSqBQ
https://man.twcc.ai/@twccdocs/doc-twnia2-main-zh/https%3A%2F%2Fman.twcc.ai%2F%40twccdocs%2Fguide-twnia2-queue-zh

查看Partition的狀態

國家實驗研究院 NATIONAL INSTITUTES OF APPLIED RESEARCH

sinfo

```
PARTITION AVAIL  TIMELIMIT  NODES  STATE NODELIST
gtest        up      30:00      2   idle gn[1001-1002]
gp1d         up 1-00:00:00      3 drain* gn[1010,1216,1226]
gp1d         up 1-00:00:00     20    mix gn[1202-1209,1211-1213,1217-1218,1220-1222,1224-1225,1227,1230]
gp1d         up 1-00:00:00     16   idle gn[1004-1009,1027-1030,1115,1201,1210,1215,1223,1228]
gp2d*        up 2-00:00:00      3 drain* gn[1010,1216,1226]
gp2d*        up 2-00:00:00     20    mix gn[1202-1209,1211-1213,1217-1218,1220-1222,1224-1225,1227,1230]
gp2d*        up 2-00:00:00     16   idle gn[1004-1009,1027-1030,1115,1201,1210,1215,1223,1228]
gp4d         up 4-00:00:00      3 drain* gn[1010,1216,1226]
gp4d         up 4-00:00:00     20    mix gn[1202-1209,1211-1213,1217-1218,1220-1222,1224-1225,1227,1230]
gp4d         up 4-00:00:00     16   idle gn[1004-1009,1027-1030,1115,1201,1210,1215,1223,1228]
express      up 4-00:00:00      3 drain* gn[1010,1216,1226]
express      up 4-00:00:00     20    mix gn[1202-1209,1211-1213,1217-1218,1220-1222,1224-1225,1227,1230]
express      up 4-00:00:00     16   idle gn[1004-1009,1027-1030,1115,1201,1210,1215,1223,1228]
```

sinfo -s

```
[u8880716@un-ln01 ~]$ sinfo -s
PARTITION AVAIL  TIMELIMIT   NODES(A/I/O/T)  NODELIST
gtest        up      30:00           0/2/0/2  gn[1001-1002]
gp1d         up 1-00:00:00       21/15/3/39  gn[1004-1010,1027-1030,1115,1201-1213,1215-1218,1220-1228,1230]
gp2d*        up 2-00:00:00       21/15/3/39  gn[1004-1010,1027-1030,1115,1201-1213,1215-1218,1220-1228,1230]
gp4d         up 4-00:00:00       21/15/3/39  gn[1004-1010,1027-1030,1115,1201-1213,1215-1218,1220-1228,1230]
express      up 4-00:00:00       21/15/3/39  gn[1004-1010,1027-1030,1115,1201-1213,1215-1218,1220-1228,1230]
```

sinfo -a 查看隱藏partition

查看Partition組態

國家實驗研究院 NATIONAL INSTITUTES OF APPLIED RESEARCH

scontrol show partition <partition_name>

```
[u8880716@cbi-lgn01 ~]$ scontrol show partition normal
PartitionName=normal
   AllowGroups=ALL DenyAccounts=gov112009,gov113008,gov113026,gov114029 AllowQos=ALL
   AllocNodes=ALL Default=NO QoS=p_normal
   DefaultTime=NONE DisableRootJobs=NO ExclusiveUser=NO GraceTime=0 Hidden=NO
   MaxNodes=2 MaxTime=2-00:00:00 MinNodes=0 LLN=NO MaxCPUsPerNode=UNLIMITED MaxCPUsPerSocket=UNLIMITED
   Nodes=hgpn[02-05,17-21]
   PriorityJobFactor=10 PriorityTier=1 RootOnly=NO ReqResv=NO OverSubscribe=NO
   OverTimeLimit=NONE PreemptMode=OFF
   State=UP TotalCPUs=1008 TotalNodes=9 SelectTypeParameters=NONE
   JobDefaults=(null)
   DefMemPerNode=UNLIMITED MaxMemPerNode=UNLIMITED
   TRES=cpu=972,mem=17100000M,node=9,billing=972,gres/gpu=72
```

```
[u8880716@cbi-lgn01 ~]$ scontrol show partition -a taide
PartitionName=taide
   AllowGroups=ALL AllowAccounts=gov112009,gov113008,gov113026,gov113080 AllowQos=ALL
   AllocNodes=ALL Default=NO QoS=p_taide
   DefaultTime=NONE DisableRootJobs=NO ExclusiveUser=NO GraceTime=0 Hidden=YES
   MaxNodes=UNLIMITED MaxTime=3-00:00:00 MinNodes=0 LLN=NO MaxCPUsPerNode=UNLIMITED MaxCPUsPerSocket=UNLIMITED
   Nodes=hgpn[02-05,17-21]
   PriorityJobFactor=199 PriorityTier=1 RootOnly=NO ReqResv=NO OverSubscribe=NO
   OverTimeLimit=NONE PreemptMode=OFF
   State=UP TotalCPUs=1008 TotalNodes=9 SelectTypeParameters=NONE
   JobDefaults=(null)
   DefMemPerNode=UNLIMITED MaxMemPerNode=UNLIMITED
   TRES=cpu=972,mem=17100000M,node=9,billing=972,gres/gpu=72
```

查看計算節點組態

國家實驗研究院 NATIONAL INSTITUTES OF APPLIED RESEARCH

`scontrol show node <node_name>`

```
[u8880716@cbi-lgn01 ~]$ scontrol show node -a hgpn20
NodeName=hgpn20 Arch=x86_64 CoresPerSocket=56
   CPUAlloc=1 CPUEfctv=108 CPUTot=112 CPULoad=17.90
   AvailableFeatures=(null)
   ActiveFeatures=(null)
   Gres=gpu:H100:8(S:0-1)
   NodeAddr=hgpn20 NodeHostName=hgpn20 Version=23.11.4
   OS=Linux 4.18.0-513.24.1.el8_9.x86_64 #1 SMP Thu Mar 14 14:20:09 EDT 2024
   RealMemory=1900000 AllocMem=1638400 FreeMem=242722 Sockets=2 Boards=1
   CoreSpecCount=4 CPUSpecList=54-55,110-111
   State=MIXED ThreadsPerCore=1 TmpDisk=0 Weight=1 Owner=N/A MCS_label=N/A
   Partitions=GTAIDE
   BootTime=2024-12-27T14:16:24 SlurmdStartTime=2025-02-07T15:53:54
   LastBusyTime=2025-02-07T15:53:54 ResumeAfterTime=None
   CfgTRES=cpu=108,mem=1900000M,billing=108,gres/gpu=8
   AllocTRES=cpu=1,mem=1600G,gres/gpu=8
   CapWatts=n/a
   CurrentWatts=0 AveWatts=0
   ExtSensorsJoules=n/a ExtSensorsWatts=0 ExtSensorsTemp=n/a
```

此行表示正在使用資源
gres/gpu表示已使用的GPU數量

查詢所有計算節點的已使用GPU數量

`scontrol show node | grep -E 'NodeName|AllocTRES'`

```
[u8880716@cbi-lgn01 ~]$ scontrol show node | grep -E 'NodeName|AllocTRES'
NodeName=hgpn02 Arch=x86_64 CoresPerSocket=56
   AllocTRES=cpu=21,mem=800G,gres/gpu=7
NodeName=hgpn03 Arch=x86_64 CoresPerSocket=56
   AllocTRES=cpu=16,mem=400G,gres/gpu=2
```

查看job

squeue

```
[u8880716@un-ln01 ~]$ squeue
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
            746268      gp4d neb_in.n u8714813 PD       0:00     26 (Resources)
            747428      gp4d decompre u5727520 PD       0:00      1 (Priority)
            747019      gp2d    2DVVM aaron900  R    6:15:32      1 gn1211
            747289      gp1d     bash mattp131  R    3:20:11      1 gn1201
            744061      gp4d    sugar caohieu9  R 1-01:58:36      1 gn1204
```

```
squeue -o "%.9i %.9P %.12j %.10u %.2t %.10M %.6D %.16R %b"
```

-o "... " 表示輸出格式

%b 表示GPU數量

```
[u8880716@un-ln01 ~]$ squeue -o "%.9i %.9P %.12j %.10u %.2t %.10M %.6D %.16R %b"
    JOBID PARTITION         NAME       USER ST       TIME  NODES NODELIST(REASON) TRES_PER_NODE
   741411      gp2d         dtpB   u5671665 PD       0:00      1 (QOSMaxGRESPerUs gpu:8
   741412      gp2d         bltB   u5671665 PD       0:00      1 (QOSMaxGRESPerUs gpu:8
   741413      gp2d        lslmB   u5671665 PD       0:00      1 (QOSMaxGRESPerUs gpu:8
   741829      gp1d         bash   s6300121  R      45:28      1           gn1206 gpu:4
   741952      gp4d bert_mlperf_   u8880716  R      26:30      2    gn[1206-1207] gpu:2
   741493      gp1d       run.sh   u8144818  R    3:01:33      1           gn1204 gpu:1
   741491      gp1d       run.sh   u8144818  R    3:04:08      1           gn1204 gpu:1
```

查看job history

```
sacct --user <user_name> -X -S 2025-06-01 -E 2025-07-10 -o
JobID,Account,User%10,Partition%10,NNodes,AllocTRES%40,State%20,Submit,Elapsed
```

-X 忽略job step

-S 起始時間

-E 結束時間

-o 輸出格式

```
[u8880716@un-ln01 ~]$ sacct --user u8880716 -X -S 2025-06-01 -E 2025-07-10 -o JobID,Account,User%10,Partition%10,NNodes,AllocTRES%40,State%20,Submit,Ela
psed
       JobID    Account       User  Partition   NNodes                                AllocTRES                State              Submit    Elapsed
------------ ---------- ---------- ---------- -------- ---------------------------------------- -------------------- ------------------- ----------
723358        gov108032   u8880716        gp1d        1 billing=4,cpu=4,gres/gpu=8,mem=720G,nod+            COMPLETED 2025-06-16T14:54:56   08:38:56
723532        gov108032   u8880716        gp1d        1 billing=4,cpu=4,gres/gpu=8,mem=720G,nod+   CANCELLED by 12409 2025-06-17T09:54:24   00:56:04
723540        gov108032   u8880716        gp1d        1 billing=4,cpu=4,gres/gpu=8,mem=720G,nod+            COMPLETED 2025-06-17T11:06:36   04:06:23
723637        gov108032   u8880716        gp1d        1 billing=4,cpu=4,gres/gpu=8,mem=720G,nod+            COMPLETED 2025-06-17T17:15:21   02:20:41
728175        gov109092   u8880716        gp4d        1 billing=4,cpu=4,gres/gpu=8,mem=720G,nod+            COMPLETED 2025-07-08T17:25:13   14:06:37
```

https://slurm.schedmd.com/sacct.html

執行方式

- 腳本
  - sbatch <slurm script>
- 互動式
  - srun
  - salloc

submit slurm script

--cpus-per-task=4 for Taiwania2

```
#!/bin/bash
#SBATCH --account=<PROJECT_ID>       # (-A) iService Project ID
#SBATCH --job-name=test               # (-J) Job name
#SBATCH --partition=development       # (-p) Slurm partition
#SBATCH --nodes=2                     # (-N) Maximum number of nodes to be allocated
#SBATCH --cpus-per-task=1             # (-c) Number of cores per MPI task
#SBATCH --ntasks-per-node=2           # Maximum number of tasks on each node
#SBATCH --time=00:30:00               # (-t) Wall time limit (days-hrs:min:sec)
#SBATCH --output=%x-%j.out            # (-o) Path to the standard output file, %x is job name, %j is job id
#SBATCH --error=%x-%j.err             # (-e) Path to the standard error file
#SBATCH --mail-type=END,FAIL          # Mail events (NONE, BEGIN, END, FAIL, ALL)
#SBATCH --mail-user=user@example.com  # Where to send mail.  Set this to your email address

module purge
module load intel/....

SIF="/path/openmpi.sif"
SINGULARITY="singularity run -B /work:/work --nv $SIF"

srun --mpi=pmix -n 4 $SINGULARITY ./hello
mpiexec ./hello
```

run.slurm

https://man.twcc.ai/@f1-manual/slurm_job_example

Submit and manager job

國家實驗研究院 NATIONAL INSTITUTES OF APPLIED RESEARCH

提交Job Script

- sbatch run.slurm

查看使用者的job 狀態

- squeue -u <username>
- squeue -j <job_id>

查看正在運行的Job 組態

- scontrol show job <job_id>

刪除 job

- scancel <job_id>

https://man.twcc.ai/@f1-manual/slurm_instructions#tip1

srun

for Nano5

```
srun -A <project> -p dev --gres=gpu:1 <command>
```

for Taiwania2

```
srun -A <project> -p gtest --nodes 1 --ntasks-per-node 1 --gres=gpu:1 <command>
```

```
[u8880716@cbi-lgn01 ~]$ srun -A GOV108032 -J test -p dev --gres=gpu:1 nvidia-smi
Fri Jul 25 15:15:02 2025
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 550.127.08             Driver Version: 550.127.08     CUDA Version: 12.4     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA H100 80GB HBM3          On  |   00000000:86:00.0 Off |                    0 |
| N/A   26C    P0             70W /  700W |       1MiB /  81559MiB |      0%      Default |
|                                         |                        |             Disabled |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI        PID   Type   Process name                              GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
```

salloc

國家實驗研究院 NATIONAL INSTITUTES OF APPLIED RESEARCH

```
salloc -A <project> -p gtest --nodes 1 --ntasks-per-node 1 --gres=gpu:1

<command> …

scancel <job_ID>
```

```
[u8880716@un-ln01 ~]$ salloc -A GOV108032 -p gtest --nodes 1 --ntasks-per-node 1 --gres=gpu:1
WARNING: When login node is DOWN, Your Job will FAIL!
WARNING: Please use `sbatch` instead :)
salloc: Granted job allocation 753642
salloc: Waiting for resource configuration
salloc: Nodes gn1002 are ready for job
[u8880716@un-ln01 ~]$ echo $SLURM_NODELIST
gn1002
[u8880716@un-ln01 ~]$ scancel 753642
salloc: Job allocation 753642 has been revoked.
[u8880716@un-ln01 ~]$ squeue -u u8880716
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
```

Slurm環境變數

```
env | grep SLURM
```

變數名稱	描述
SLURM_JOB_ID	Job ID
SLURM_JOB_NAME	Job name
SLURM_JOB_ACCOUNT	Project id
SLURM_JOB_NUM_NODES SLURM_NNODES	Number of nodes allocated to job
SLURM_JOB_NODELIST SLURM_NODELIST	Nodes assigned to job
SLURM_NTASKS_PER_NODE	Number of tasks requested per node.

https://hpcc.umd.edu/hpcc/help/slurmenv.html

Slurm環境變數

env |grep SLURM

```
[u8880716@cbi-lgn01 ~]$ srun -A GOV108032 -p dev --gres=gpu:1 env |grep SLURM
srun: job 37745 queued and waiting for resources
srun: job 37745 has been allocated resources
SLURM_CONF=/etc/slurm/slurm.conf
SLURM_PRIO_PROCESS=0
SLURM_UMASK=0022
SLURM_CLUSTER_NAME=hpc
SLURM_SUBMIT_DIR=/home/u8880716
SLURM_SUBMIT_HOST=cbi-lgn01
SLURM_JOB_NAME=env
SLURM_JOB_CPUS_PER_NODE=1
SLURM_MEM_PER_NODE=204800
SLURM_NTASKS=1
SLURM_NPROCS=1
SLURM_DISTRIBUTION=cyclic
SLURMD_DEBUG=2
SLURM_JOB_ID=37745
SLURM_JOBID=37745
SLURM_STEP_ID=0
SLURM_STEPID=0
SLURM_NNODES=1
SLURM_JOB_NUM_NODES=1
SLURM_NODELIST=hgpn04
SLURM_JOB_PARTITION=dev
SLURM_TASKS_PER_NODE=1
SLURM_JOB_UID=12409
SLURM_JOB_USER=u8880716
SLURM_JOB_GID=18361
SLURM_JOB_GROUP=TRI1072239
SLURM_JOB_ACCOUNT=gov108032
SLURM_JOB_QOS=normal
SLURM_JOB_NODELIST=hgpn04
SLURM_STEP_NODELIST=hgpn04
SLURM_STEP_NUM_NODES=1
SLURM_STEP_NUM_TASKS=1
SLURM_STEP_TASKS_PER_NODE=1
SLURM_STEP_LAUNCHER_PORT=47743
SLURM_SRUN_COMM_PORT=47743
SLURM_SRUN_COMM_HOST=172.21.101.1
SLURM_GPUS_ON_NODE=1
SLURM_STEP_GPUS=3
SLURM_TOPOLOGY_ADDR=ibsw1.hgpn04
SLURM_TOPOLOGY_ADDR_PATTERN=switch.node
SLURM_CPUS_ON_NODE=1
SLURM_CPU_BIND=quiet,mask_cpu:0x00000000000000000000000002
SLURM_CPU_BIND_LIST=0x00000000000000000000000002
SLURM_CPU_BIND_TYPE=mask_cpu:
SLURM_CPU_BIND_VERBOSE=quiet
SLURM_JOB_END_TIME=1753681600
SLURM_JOB_START_TIME=1753674400
SLURM_TASK_PID=1834617
SLURM_NODEID=0
SLURM_PROCID=0
SLURM_LOCALID=0
SLURM_LAUNCH_NODE_IPADDR=172.21.101.1
SLURM_GTIDS=0
SLURMD_NODENAME=hgpn04
```

環境管理系統：Lmod

Lmod幫助使用者管理在共享 HPC 環境中可用的各種軟體和應用程式。 由於 HPC 叢集通常會安裝許多不同的軟體套件，包括不同版本的編譯器、函式庫、科學應用程式等等，直接管理這些軟體可能會變得非常複雜，而且容易產生衝突Lmod就是為了簡化這個複雜性而設計的。

Nano5

```
------------------------------------------ /work/HPC_software/LMOD/public/modulefiles ------------------------------------------
   cmake/3.24.2    hwloc/2.7.2    miniconda3/24.11.1    os    pmix/4.2.9    singularity/3.7.1

------------------------------------------ /work/HPC_software/LMOD/nvidia/modulefiles ------------------------------------------
   cuda/11.6    cuda/12.4             (D)    nvhpc/24.7          openmpi/5.0.5
   cuda/12.2    nvhpc-hpcx-cuda12/24.7       openmpi/4.1.6 (D)    ucx/1.16.0

------------------------------------------ /work/HPC_software/LMOD/intel/modulefiles ------------------------------------------
   advisor/latest                      compiler/2024.2.1      (D)    ifort/latest                      mkl/2024.2    (D)
   advisor/2024.2                (D)   compiler32/latest             ifort/2024.2.1              (D)   mkl32/latest
   ccl/latest                          compiler32/2024.2.1    (D)    ifort32/latest                    mkl32/2024.2  (D)
   ccl/2021.13.1                 (D)   debugger/latest               ifort32/2024.2.1            (D)   mpi/latest
   compiler-intel-llvm/latest          debugger/2024.2.1      (D)    intel_ipp_ia32/latest             mpi/2021.13   (D)
   compiler-intel-llvm/2024.2.1  (D)   dev-utilities/latest          intel_ipp_ia32/2021.12      (D)   tbb/latest
   compiler-intel-llvm32/latest        dev-utilities/2024.2.0 (D)    intel_ipp_intel64/latest          tbb/2021.13   (D)
   compiler-intel-llvm32/2024.2.1 (D)  dnnl/latest                   intel_ipp_intel64/2021.12   (D)   tbb32/latest
   compiler-rt/latest                  dnnl/3.5.0             (D)    intel_ippcp_ia32/latest           tbb32/2021.13 (D)
   compiler-rt/2024.2.1          (D)   dpct/latest                   intel_ippcp_ia32/2021.12    (D)   vtune/latest
   compiler-rt32/latest                dpct/2024.2.0          (D)    intel_ippcp_intel64/latest        vtune/2024.2  (D)
   compiler-rt32/2024.2.1        (D)   dpl/latest                    intel_ippcp_intel64/2021.12 (D)
   compiler/latest                     dpl/2022.6             (D)    mkl/latest
```

Taiwania 2

```
------------------------------------------ /work/opt/t2_r8/modulefiles/tools ------------------------------------------
   cmake/3.23.2    git/2.46.2                        openmpi/5.0.2_ucx1.14.1_cuda12.3 (D)
   ffmpeg/7.1      openmpi/4.1.6_ucx1.14.1_cuda12.3  s5cmd/2.2.2

------------------------------------------ /work/opt/t2_r8/modulefiles/nvhpc ------------------------------------------
   nvhpc-22.11_hpcx-2.12_cuda-11.0    nvhpc-23.9_hpcx-2.16_cuda-12.2    nvhpc-24.11_hpcx-2.20_cuda-12.6
   nvhpc-22.11_hpcx-2.12_cuda-11.8    nvhpc-24.11_hpcx-2.14_cuda-11.8

------------------------------------------ /work/opt/t2_r8/modulefiles/cuda ------------------------------------------
   cuda/10.2    cuda/11.4    cuda/11.7    cuda/12.3    cuda/12.8 (D)

------------------------------------------ /work/opt/t2_r8/modulefiles/centos7 ------------------------------------------
   amber/18/multigpus        gcc7/7.3.1    intel/2020                (D)    rclone                        zsh/5.9
   amber/18/singlegpu (D)    gcc8/8.3.1    java/11.0.18                     schrodinger/sch2022-3
   amber/20/multigpus        gcc9/9.3.1    miniconda2/conda4.8.4_py2.7      schrodinger/sch2023-2
   amber/20/singlegpu (D)    gsl/2.7       miniconda3/conda24.5.0_py3.9     schrodinger/sch2023-4 (D)
   gcc10/10.2.1              intel/2018    miniforge/24.7.1-2               szip/2.1.1

  Where:
   D:  Default Module
```

Modules command

查看可安裝的軟體 module avail

載入軟體 modude load <package>

查看載入軟體 module list

清除軟體 module purge

Singularity container

- Singularity 是一種專為高效能運算 (HPC) 環境設計的容器化技術，其主要目的是將應用程式及其所有依賴項（如程式庫、環境設定等）打包成一個獨立的容器映像檔，從而實現跨平台、跨叢集運算環境的一致性與可重現性。
- **環境封裝**：Singularity 可將一個完整的運行環境打包成單一映像檔，使用者無需在每個運算節點上手動安裝軟體與依賴，就能確保應用程式在不同系統間執行時具有一致性。
- **跨平台可攜**：使用者可以在個人電腦上建立容器，並將其部署至大規模 HPC 叢集、雲端或其他運算平台，確保相同程式碼在不同環境中得到一致的執行結果。
- **權限一致**：Singularity 的設計理念是“用戶在容器內的身份與在宿主系統上相同”，因此不需要 root 權限來執行容器，降低了安全風險，也符合多使用者共享環境的要求。
- **輕量化部署**：與 Docker 等容器技術相比，Singularity 不依賴於守護進程（daemon），能更好地與 HPC 系統的資源調度器（例如 SLURM）整合，使得容器化工作流程能夠順利提交到叢集任務中執行。
- **MPI 與 GPU 支援**：Singularity 能夠透過綁定 (binding) 主機上的網路設備、MPI 函式庫以及 GPU 等硬體資源，使得容器內的應用能夠利用 HPC 叢集的高速網路與專用加速器，達到接近裸機的效能。
- **再現性**：科學研究中經常需要重現先前的計算結果。Singularity 可確保運行環境與軟體版本固定，使得實驗結果更具再現性，利於學術交流與結果驗證。

Singularity映像檔

國家實驗研究院 NATIONAL INSTITUTES OF APPLIED RESEARCH

Taiwania2 預設安裝的映像檔

- `ls /work/TWCC_cntr`

```
[u8880716@un-ln01 pytorch-imagenet]$ ls /work/TWCC_cntr
cuda_11.5.1-cudnn8-devel-centos8.sif          pytorch_20.02-py3.sif          tensorflow_20.02-tf2-py3.sif
cuda_11.5.1-cudnn8-devel-ubuntu20.04.sif      pytorch_20.09-py3_horovod.sif  tensorflow_20.09-tf1-py3.sif
cuda_11.5.1-cudnn8-runtime-centos8.sif        pytorch_20.09-py3.sif          tensorflow_20.09-tf2-py3.sif
cuda_11.5.1-cudnn8-runtime-ubuntu20.04.sif    pytorch_21.06-py3_horovod.sif  tensorflow_21.11-tf1-py3.sif
cuda_latest.sif                               pytorch_21.09-py3.sif          tensorflow_21.11-tf2-py3.sif
FFM_assets                                    pytorch_21.11-py3_horovod.sif  tensorflow_22.01-tf2-py3.sif
gnu-openmpi-cuda-lammps-kmo.sif               pytorch_21.11-py3.sif          tensorflow_22.11-tf1-py3.sif
hpc-benchmarks:24.03.sif                      pytorch_22.01-py3.sif          tensorflow_22.11-tf2-py3.sif
mxnet_21.08-py3.sif                           pytorch_22.11-py3.sif          tensorflow_23.02-tf1-py3.sif
mxnet_21.09-py3.sif                           pytorch_23.02-py3_horovod.sif  tensorflow_23.02-tf2-py3.sif
nvhpc_22.1-devel-cuda11.5-centos7.sif         pytorch_23.02-py3.sif          tensorflow_23.05-tf2-py3.sif
nvhpc_22.1-devel-cuda11.5-ubuntu20.04.sif     pytorch_23.05-py3.sif          tensorflow_23.08-tf2-py3.sif
nvhpc_22.1-runtime-cuda11.5-centos7.sif       pytorch_23.08-py3.sif          tensorflow_23.11-tf2-py3.sif
nvhpc_22.1-runtime-cuda11.5-ubuntu20.04.sif   pytorch_23.11-py3.sif          tensorflow_24.02-tf2-py3.sif
pytorch_19.01-py3.sif                         pytorch_24.02-py3.sif          tensorflow_24.05-tf2-py3.sif
pytorch_20.02-py3_horovod_fastai.sif          pytorch_24.05-py3.sif          user
pytorch_20.02-py3_horovod.sif                 README
pytorch_20.02-py3_mpi4py.sif                  tensorflow_20.02-tf1-py3.sif
```

自行下載

- `singularity pull tensorflow_21.10-tf2-py3.sif docker://nvcr.io/nvidia/tensorflow:21.10-tf2-py3`

執行 Singularity容器

從Docker hub下載映像檔

- `singularity pull <image.sif> docker://nvcr.io/nvidia/pytorch:23.08-py3`

運行容器

- 背景執行 `singularity exec/run --nv <image.sif> <command>`
- 互動執行 `singularity run/shell --nv <image.sif>`

```
$ singularity shell instance://instance3

Singularity>
```

- --nv 允許容器使用GPU
- 引用/work目錄 `singularity exec/run -B /work:/work --nv <image.sif> <command>`
- 列出python package
  - singularity exec/run --nv <image.sif> pip list

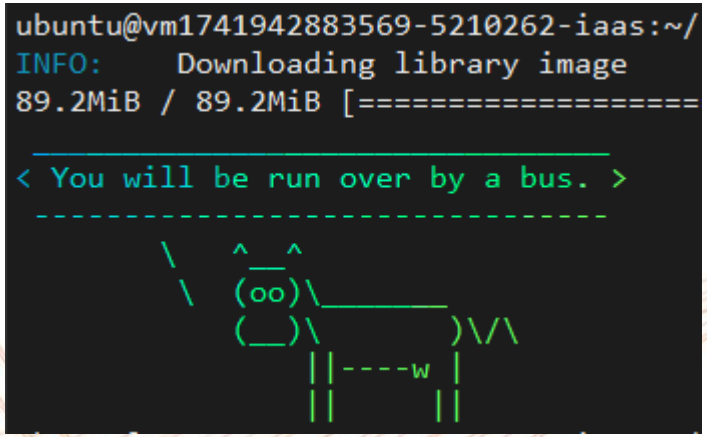
建立客製化容器映像檔 (實作)

1. 建立客製化容器映像檔需要sudo權限
2. 另尋Linux電腦 (例如TWCC VM, WSL, VirtualBox)建立客製化容器映像檔
3. 安裝相關依賴庫、Go語言與Singularity
   - 注意Go語言與Singularity版本有搭配
4. 實作版本
   - Linux Ubuntu 24.04
   - Docker 28.3.2 and Nvidia container toolkit 1.17.8 (非必要)
   - Go 1.23.6
   - Singularity: Apptainer 1.4.1
   - https://github.com/apptainer/apptainer/blob/main/INSTALL.md

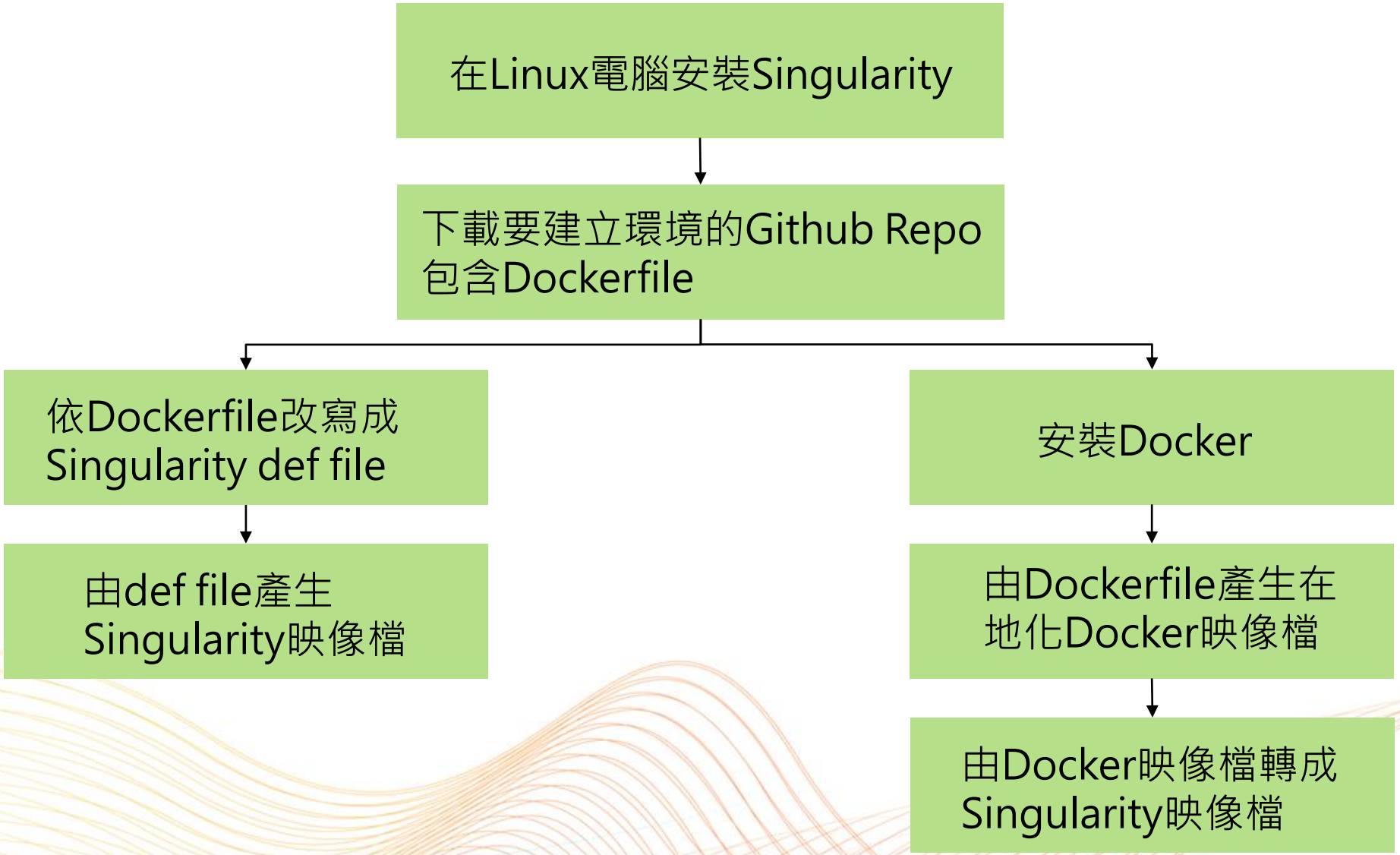
Install Go & Singularity

```
# Ensure repositories are up-to-date
sudo apt-get update
# Install debian packages for dependencies
sudo apt-get install -y \
    build-essential \
    libseccomp-dev \
    uidmap \
    fakeroot \
    cryptsetup \
    tzdata \
    dh-apparmor \
    libsubid-dev \
    pkg-config \
    curl wget git

# Install GO
export GOVERSION=1.23.6 OS=linux ARCH=amd64  # change this as you need

wget -O /tmp/go${GOVERSION}.${OS}-${ARCH}.tar.gz \
  https://dl.google.com/go/go${GOVERSION}.${OS}-${ARCH}.tar.gz

sudo tar -C /usr/local -xzf /tmp/go${GOVERSION}.${OS}-${ARCH}.tar.gz

# add /usr/local/go/bin to the PATH environment variable
echo 'export PATH=$PATH:/usr/local/go/bin' >> ~/.bashrc
source ~/.bashrc
```

```
# Install singulairty
git clone https://github.com/apptainer/apptainer.git
cd apptainer
git checkout v1.4.1

# Compiling Singularity
./mconfig
cd $(/bin/pwd)/builddir
make -j 4
sudo make install

singularity --version
singularity run library://godlovedc/funny/lolcow
```



建立客製化容器映像檔的步驟



建立定義檔Singularity Definition file

國家實驗研究院 NATIONAL INSTITUTES OF APPLIED RESEARCH

vi <definition>.def

實作：
在Pytorch映像檔安裝Horovod
製作有Deepspeed的Singularity映像檔

```
Bootstrap: library
From: ubuntu:18.04
Stage: build

%setup
    touch /file1
    touch ${SINGULARITY_ROOTFS}/file2

%files
    /file1
    /file1 /opt

%environment
    export LISTEN_PORT=12345
    export LC_ALL=C

%post
    apt-get update && apt-get install -y netcat
    NOW=`date`
    echo "export NOW=\"${NOW}\"" >> $SINGULARITY_ENVIRONMENT

%runscript
    echo "Container was created $NOW"
    echo "Arguments received: $*"
    exec echo "$@"
```

從Docker hub取得映像檔

建立主機端和容器的檔案、資料夾

複製主機端的檔案、資料夾到容器

設定環境變數

執行安裝軟體指令

啟動容器時，執行程式

https://docs.sylabs.io/guides/3.7/user-guide/definition_files.html
HowTo：建立 TWNIA2 容器

建立映像檔

國家實驗研究院 NATIONAL INSTITUTES OF APPLIED RESEARCH

方法一：By singularity definition file

```
sudo singularity build <image>.sif <definition>.def
```

方法二： By docker local image

1. `docker build -t <repository>:<tag> .`
2. `sudo singularity build <image>.sif docker-daemon://<repository>:<tag>`

方法三： By docker local image and compressed file

1. `docker build -t <repository>:<tag> .`
2. `docker save <repository>:<tag> -o <image>.tar`
3. `sudo singularity build <image>.sif docker-archive://$(pwd)/<name>.tar`

```
ubuntu@vm1692770172744-3864139-iaas:~$ docker image list
REPOSITORY                TAG               IMAGE ID       CREATED          SIZE
mlperf-nvidia             language_model    7b697e4dcb2b   24 hours ago     22.5GB
llama-factory             latest            a334a786d902   3 months ago     22.7GB
nvcr.io/nvidia/pytorch    24.01-py3         8470a68886ff   7 months ago     22GB
nvcr.io/nvidia/pytorch    23.04-py3         b4428941db4f   16 months ago    20.4GB
```

Horovod

Use Horovod on Keras(tf2.x)

```
import horovod.tensorflow.keras as hvd

hvd.init() # Horovod: initialize Horovod.

# Horovod: pin GPU to be used to process local rank (one GPU per process)分配 GPU 資源
gpus = tf.config.experimental.list_physical_devices('GPU')
tf.config.experimental.set_visible_devices(gpus[hvd.local_rank()], 'GPU')

scaled_lr = 0.001 * hvd.size() # Horovod: adjust learning rate based on number of GPUs.

opt = hvd.DistributedOptimizer(opt) # Horovod: add Horovod DistributedOptimizer.包裝原始優化器

# Horovod: Specify `experimental_run_tf_function=False` to ensure TensorFlow
# uses hvd.DistributedOptimizer() to compute gradients.
model.compile(loss=tf.losses.SparseCategoricalCrossentropy(),
              optimizer=opt,
              metrics=['accuracy'],
              experimental_run_tf_function=False)
```

https://github.com/horovod/horovod/blob/master/examples/tensorflow2/tensorflow2_keras_mnist.py
https://horovod.readthedocs.io/en/latest/keras.html

Use Horovod on Keras(tf2.x) (cont.)

國家實驗研究院 NATIONAL INSTITUTES OF APPLIED RESEARCH

```
callbacks = [
        # Horovod: broadcast initial variable states from rank 0 to all other processes.同步初始變數狀態到所有工作節點
        # This is necessary to ensure consistent initialization of all workers when
        # training is started with random weights or restored from a checkpoint.
        hvd.callbacks.BroadcastGlobalVariablesCallback(0),

        # Horovod: average metrics among workers at the end of every epoch.存儲模型檔案僅限於主節點
        # Note: This callback must be in the list before the ReduceLROnPlateau,
        # TensorBoard or other metrics-based callbacks.
        hvd.callbacks.MetricAverageCallback(),

        # Horovod: using `lr = 1.0 * hvd.size()` from the very beginning leads to worse final
        # accuracy. Scale the learning rate `lr = 1.0` ---> `lr = 1.0 * hvd.size()` during
        # the first three epochs. See https://arxiv.org/abs/1706.02677 for details.
        hvd.callbacks.LearningRateWarmupCallback(initial_lr=scaled_lr, warmup_epochs=3, verbose=1),
    ]

    # Horovod: save checkpoints only on worker 0 to prevent other workers from corrupting them.如果是主節點，才保存檢查點
    if hvd.rank() == 0:
        callbacks.append(tf.keras.callbacks.ModelCheckpoint('./checkpoint-{epoch}.h5'))

    # Horovod: write logs on worker 0.
    verbose = 1 if hvd.rank() == 0 else 0

    # Train the model.
    # Horovod: adjust number of steps based on number of GPUs.
    model.fit(dataset, steps_per_epoch=500 // hvd.size(), callbacks=callbacks, epochs=24, verbose=verbose)
```

Launch Horovod with slrum script

國家實驗研究院 NATIONAL INSTITUTES OF APPLIED RESEARCH

run_horovod.slurm

ntasks-per-node, gres=gpu 數量要相同

```
#!/bin/bash
#SBATCH -A <Project_id>           # iService Project id
#SBATCH -J horovod              # job name
#SBATCH -p gtest                # partition gtest gp1d, gpNCHC_LLM
#SBATCH --nodes=2                # Maximum number of nodes to be allocated
#SBATCH --cpus-per-task=4        # Number of cores per MPI task
#SBATCH --ntasks-per-node=2      # Number of MPI tasks (i.e. processes)
#SBATCH --gres=gpu:2
#SBATCH -o %x_%j.out             # Path to the standard output file

SIF="/work/TWCC_cntr/tensorflow_21.11-tf2-py3.sif"
SINGULARITY="singularity run -B /work:/work --nv $SIF"

CMD="python xxxx.py"
RUN="srun $SINGULARITY $CMD"
echo $RUN; $RUN
```

sbatch run_horovod.slurm

http://www.idris.fr/eng/jean-zay/gpu/jean-zay-gpu-hvd-tf-multi-eng.html

Example 1 : test_horovod.py

國家實驗研究院 NATIONAL INSTITUTES OF APPLIED RESEARCH

test_horovod.py

```
FRAMEWORK="TF" #TF or TORCH

if FRAMEWORK=="TF":
    import tensorflow as tf
    import horovod.keras as hvd
    print("List of TF visible physical GPUs : ", tf.config.experimental.list_physical_devices("GPU"))
elif FRAMEWORK=="TORCH":
    import torch
    import horovod.torch as hvd
    print("List of PyTorch visible physical GPUs : ", torch.cuda.device_count())
else:
    raise ValueError(f"FRAMEWORK '{FRAMEWORK}' NOT UNDERSTODD")

hvd.init()

msg = f"MPI_size = {hvd.size()}, MPI_rank = {hvd.rank()}, MPI_local_size = {hvd.local_size()},  MPI_local_rank = {hvd.local_rank()}"

try:
    import platform #
    msg += f" platform = {platform.node()}"
except ImportError:
    pass

print(msg)
```

https://ulhpc-tutorials.readthedocs.io/en/latest/deep_learning/horovod/app/test_horovod.py

Run test_horovod.py on Taiwania2

run_horovod.slurm

```
#!/bin/bash
#SBATCH -A <Project_id>           # iService Project id
#SBATCH -J horovod             # job name
#SBATCH -p gtest               # partition gtest gp1d, gpNCHC_LLM
#SBATCH --nodes=2               # Maximum number of nodes to be allocated
#SBATCH --cpus-per-task=4       # Number of cores per MPI task
#SBATCH --ntasks-per-node=2     # Number of MPI tasks (i.e. processes)
#SBATCH --gres=gpu:2
#SBATCH -o %x_%j.out            # Path to the standard output file

SIF="/work/TWCC_cntr/tensorflow_21.11-tf2-py3.sif"
SINGULARITY="singularity run -B /work:/work --nv $SIF"

CMD="python test_horovod.py"
RUN="srun $SINGULARITY $CMD"
echo $RUN; $RUN
```

sbatch run_horovod.slurm

```
List of TF visible physical GPUs :  [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU'), Physica
lDevice(name='/physical_device:GPU:1', device_type='GPU')]
MPI_size = 4, MPI_rank = 0, MPI_local_size = 2,  MPI_local_rank = 0 platform = gn1001.twcc.ai
List of TF visible physical GPUs :  [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU'), Physica
lDevice(name='/physical_device:GPU:1', device_type='GPU')]
MPI_size = 4, MPI_rank = 1, MPI_local_size = 2,  MPI_local_rank = 1 platform = gn1001.twcc.ai
List of TF visible physical GPUs :  [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU'), Physica
lDevice(name='/physical_device:GPU:1', device_type='GPU')]
MPI_size = 4, MPI_rank = 2, MPI_local_size = 2,  MPI_local_rank = 0 platform = gn1002.twcc.ai
List of TF visible physical GPUs :  [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU'), Physica
lDevice(name='/physical_device:GPU:1', device_type='GPU')]
MPI_size = 4, MPI_rank = 3, MPI_local_size = 2,  MPI_local_rank = 1 platform = gn1002.twcc.ai
```

Example 2 : Run tensorflow2_keras_mnist.py

https://github.com/horovod/horovod/blob/master/examples/tensorflow2/tensorflow2_keras_mnist.py

```
#!/bin/bash
#SBATCH -A <Project_id>           # iService Project id
#SBATCH -J horovod             # job name
#SBATCH -p gp1d               # partition gtest gp1d, gpNCHC_LLM
#SBATCH --nodes=2               # Maximum number of nodes to be allocated
#SBATCH --cpus-per-task=4       # Number of cores per MPI task
#SBATCH --ntasks-per-node=2     # Number of MPI tasks (i.e. processes)
#SBATCH --gres=gpu:2
#SBATCH -o %x_%j.out            # Path to the standard output file

#export UCX_NET_DEVICES=mlx5_0:1 # connect with mellanox infiniband
#export UCX_IB_GPU_DIRECT_RDMA=1 # enable RDMA

SIF="/work/$(whoami)/image/tensorflow_21.05-tf2-py3.sif"
SINGULARITY="singularity run -B /work:/work --nv $SIF"

CMD="python tensorflow2_keras_mnist.py"
RUN="srun $SINGULARITY $CMD"
echo $RUN; $RUN
```

```
stabch run_horovod.slurm
```

```
2025-07-30 16:28:57.744711: I tensorflow/stream_executor/cuda/cuda_dnn.cc:381] Loaded cuDNN version 8300
2025-07-30 16:28:57.769675: I tensorflow/stream_executor/cuda/cuda_dnn.cc:381] Loaded cuDNN version 8300
2025-07-30 16:28:57.782359: I tensorflow/stream_executor/cuda/cuda_dnn.cc:381] Loaded cuDNN version 8300
2025-07-30 16:28:57.816153: I tensorflow/stream_executor/cuda/cuda_dnn.cc:381] Loaded cuDNN version 8300
Epoch 1/24
125/125 [==============================] - 17s 9ms/step - loss: 0.4130 - accuracy: 0.8736
Epoch 2/24
125/125 [==============================] - 1s 5ms/step - loss: 0.0977 - accuracy: 0.9704
Epoch 3/24
125/125 [==============================] - 1s 5ms/step - loss: 0.0809 - accuracy: 0.9748
```

Use Horovod on Pytorch

國家實驗研究院 NATIONAL INSTITUTES OF APPLIED RESEARCH

```
import torch
import horovod.torch as hvd

hvd.init()

if args.cuda:
    torch.cuda.set_device(hvd.local_rank()) # Horovod: pin GPU to local rank.

# Horovod: use DistributedSampler to partition the training data.
train_sampler = torch.utils.data.distributed.DistributedSampler(
                train_dataset, num_replicas=hvd.size(), rank=hvd.rank())

# By default, Adasum doesn't need scaling up learning rate.
lr_scaler = hvd.size()

# Add Horovod Distributed Optimizer
optimizer = hvd.DistributedOptimizer(optimizer, named_parameters=model.named_parameters())

# Broadcast parameters from rank 0 to all other processes.
hvd.broadcast_parameters(model.state_dict(), root_rank=0)
```

https://horovod.readthedocs.io/en/stable/pytorch.html

製作映像檔：在Pytorch映像檔安裝Horovod

國家實驗研究院 NATIONAL INSTITUTES OF APPLIED RESEARCH

pytorch_horovod.def

```
BootStrap: docker
From: nvcr.io/nvidia/pytorch:23.01-py3
Stage: build

%environment
    export HOROVOD_GPU=CUDA
    export HOROVOD_GPU_OPERATIONS=NCCL
    export HOROVOD_NCCL_LINK=SHARED
    export HOROVOD_WITHOUT_GLOO=1
    export HOROVOD_WITH_MPI=1
    export HOROVOD_WITH_PYTORCH=1
    export HOROVOD_WITHOUT_TENSORFLOW=1
    export HOROVOD_WITHOUT_MXNET=1
%post
    pip install --no-cache-dir horovod==0.27.0
```

Release date

pytorch:23.01	Feb. 1 2023
Horovod 0.27.0	Feb. 2 2023

```
sudo singularity build pytorch_23.01-py3_horovod.sif pytorch_horovod.def
```

將映像檔傳送到TWCC HFS

- `sftp <username>@ln01.twcc.ai`
- `sftp> put pytorch_20.09-py3_horovod.sif`

https://man.twcc.ai/@twccdocs/doc-twnia2-main-zh/https%3A%2F%2Fman.twcc.ai%2F%40twccdocs%2Fhowto-twnia2-run-parallel-job-container-zh

Example 3 : Run pytorch_mnist.py

https://github.com/horovod/horovod/blob/master/examples/pytorch/pytorch_mnist.py

```
#!/bin/bash
#SBATCH -A <Project_id>           # iService Project id
#SBATCH -J horovod            # job name
#SBATCH -p gtest              # partition gtest gp1d, gpNCHC_LLM
#SBATCH --nodes=2              # Maximum number of nodes to be allocated
#SBATCH --cpus-per-task=4      # Number of cores per MPI task
#SBATCH --ntasks-per-node=2    # Number of MPI tasks (i.e. processes)
#SBATCH --gres=gpu:2
#SBATCH -o %x_%j.out           # Path to the standard output file

export CUDA_VISIBLE_DEVICES=0,1,2,3,4,5,6,7

SIF="/work/$(whoami)/image/pytorch_23.01-py3_horovod.sif"
SINGULARITY="singularity run -B /work:/work --nv $SIF"

CMD="python pytorch_mnist.py"
RUN="srun $SINGULARITY $CMD"
echo $RUN; $RUN
```

```
stabch run_horovod.slurm
```

Pytorch DDP

Use DDP on Pytorch

```
import torch.multiprocessing as mp # 啟用多進程
from torch.utils.data.distributed import DistributedSampler
from torch.nn.parallel import DistributedDataParallel as DDP
from torch.distributed import init_process_group, destroy_process_group
import os

def ddp_setup():
    torch.cuda.set_device(int(os.environ["LOCAL_RANK"]))
    init_process_group(backend="nccl") # 初始化平行運算

def prepare_dataloader(dataset: Dataset, batch_size: int):
    return DataLoader(
        dataset,
        batch_size=batch_size,
        pin_memory=True,
        shuffle=False,
        sampler=DistributedSampler(dataset) # 數據平行化
    )

class Trainer:
   def __init__(
        self.gpu_id = int(os.environ["LOCAL_RANK"])
        self.model = model.to(self.gpu_id) # 將模型分配到指定 GPU
        self.model = DDP(self.model, device_ids=[self.gpu_id]) # 包裝為 DDP 模型

   def _run_epoch(self, epoch):
        b_sz = len(next(iter(self.train_data))[0])
        print(f"[GPU{self.gpu_id}] Epoch {epoch} | Batchsize: {b_sz} | Steps: {len(self.train_data)}")
        self.train_data.sampler.set_epoch(epoch) # 訓練前，進行資料洗牌，GPU會讀到不同的資料

def main(save_every: int, total_epochs: int, batch_size: int, snapshot_path: str = "snapshot.pt"):
    ddp_setup()
    dataset, model, optimizer = load_train_objs()
    train_data = prepare_dataloader(dataset, batch_size)
    trainer = Trainer(model, train_data, optimizer, save_every, snapshot_path)
    trainer.train(total_epochs)
    destroy_process_group() # 結束平行運算
```

https://github.com/pytorch/examples/tree/main/distributed/ddp-tutorial-series

single_gpu.py 適用一個GPU

multigpu.py 適用一機多卡

multigpu_torchrun.py 適用一機多卡

multinode.py 適用多機多卡

Launch Pytorch DDP

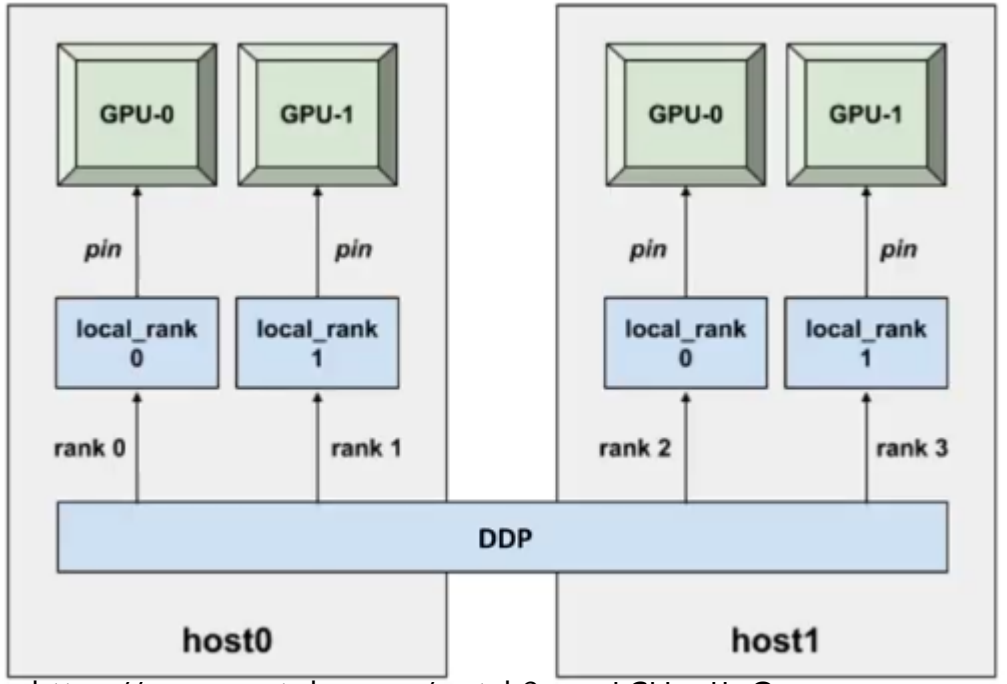
單節點

```
torchrun --nproc_per_node=2 YOUR_TRAINING_SCRIPT.py (--arg1 --arg2 --arg3)
```

多節點方法一：Elastic launch (適用Slurm叢集技統)

```
torchrun
    --nnodes=NUM_NODES 節點數量
    --nproc-per-node=TRAINERS_PER_NODE 每個節點的GPU數量
    --max-restarts=NUM_ALLOWED_FAILURES 容錯次數(不常用)
    --rdzv-id=JOB_ID 整數值
    --rdzv-backend=c10d
    --rdzv-endpoint=HOST_NODE_ADDR 主節點IP
    YOUR_TRAINING_SCRIPT.py (--arg1 ... train script args...)
```

Launch Pytorch DDP (cont.)



https://www.youtube.com/watch?v=azLCUayJJoQ

多節點方法二 (多台電腦組成的叢集系統)

Node 1

```
torchrun --nproc-per-node=NUM_GPUS_YOU_HAVE
         --nnodes=2 --node-rank=0 --master-addr="192.168.1.1"
         --master-port=1234 YOUR_TRAINING_SCRIPT.py (--arg1 --arg2 --arg3)
```

Node 2

```
torchrun --nproc-per-node=NUM_GPUS_YOU_HAVE
         --nnodes=2 --node-rank=1 --master-addr="192.168.1.1"
         --master-port=1234 YOUR_TRAINING_SCRIPT.py (--arg1 --arg2 --arg3)
```

Example 4 : Pytorch ddp tutorial series

TWCC CCS

image: pytorch-22.02-py3:latest

Spec: 2 GPU + 08 cores + 120GB memory + 60GB share memory

```
export OMP_NUM_THREADS=1
torchrun --nproc_per_node=2 multigpu_torchrun.py 10 10
```

https://github.com/pytorch/examples/tree/main/distributed/ddp-tutorial-series

single_gpu.py 適用一個GPU

multigpu.py 適用一機多卡

multigpu_torchrun.py 適用一機多卡

multinode.py 適用多機多卡

Example 4 : Pytorch DDP

執行一個計算節點

sbatch run_pytorchddp.slurm

```
#!/bin/bash
#SBATCH -A <Project_id>           # iService Project id
#SBATCH -J ddp_1x2GPU            # job name
#SBATCH -p gp1d               # partition gtest gp1d, gpNCHC_LLM
#SBATCH --nodes=1               # Maximum number of nodes to be allocated
#SBATCH --cpus-per-task=4       # Number of cores per MPI task
#SBATCH --ntasks-per-node=1     # Number of MPI tasks (i.e. processes)
#SBATCH --gres=gpu:2
#SBATCH -o %x_%j.out            # Path to the standard output file

export CUDA_VISIBLE_DEVICES=0,1 #,2,3,4,5,6,7
export OMP_NUM_THREADS=1

SIF="/work/TWCC_cntr/pytorch_22.01-py3.sif"
SINGULARITY="singularity run -B /work:/work --nv $SIF"

CMD="torchrun --nproc_per_node=2 multigpu_torchrun.py 10 10" # 1 node

echo "srun $SINGULARITY $CMD"
srun $SINGULARITY $CMD
```

用方法一跑多節點 Elastic launch

國家實驗研究院 NATIONAL INSTITUTES OF APPLIED RESEARCH

```
#!/bin/bash
#SBATCH -A <Project_id>           # iService Project id
#SBATCH -J ddp_2x2GPU          # job name
#SBATCH -p gp1d               # partition gtest gp1d, gpNCHC_LLM
#SBATCH --nodes=2               # Maximum number of nodes to be allocated
#SBATCH --cpus-per-task=4       # Number of cores per MPI task
#SBATCH --ntasks-per-node=1     # Number of MPI tasks (i.e. processes)
#SBATCH --gres=gpu:2
#SBATCH -o %x_%j.out            # Path to the standard output file

nodes=( $( scontrol show hostnames $SLURM_JOB_NODELIST ) )
nodes_array=($nodes)
head_node=${nodes_array[0]}
head_node_ip=$(srun --nodes=1 --ntasks=1 -w "$head_node" hostname --ip-address)

export CUDA_VISIBLE_DEVICES=0,1 # ,2,3,4,5,6,7
export OMP_NUM_THREADS=1

SIF="/work/TWCC_cntr/pytorch_22.01-py3.sif"
SINGULARITY="singularity run -B /work:/work --nv $SIF"

TORCHRUN="torchrun \
--nnodes $SLURM_JOB_NUM_NODES \
--nproc_per_node=2 \
--rdzv_id $RANDOM \
--rdzv_backend c10d \
--rdzv_endpoint $head_node_ip \
"
CMD="$TORCHRUN multinode.py 10 10"

echo "srun $SINGULARITY $CMD"
srun $SINGULARITY $CMD
```

sbatch run_pytorchddp.slurm

用第二個方法跑多節點

run_multinodes.slurm

```
#!/bin/bash
#SBATCH -A <Project_id>           # iService Project id
#SBATCH -J ddp_2x2GPU              # job name
#SBATCH -p gp1d                 # partition gtest gp1d, gpNCHC_LLM
#SBATCH --nodes=2               # Maximum number of nodes to be allocated
#SBATCH --cpus-per-task=4       # Number of cores per MPI task
#SBATCH --ntasks-per-node=1     # Number of MPI tasks (i.e. processes)
#SBATCH --gres=gpu:2
#SBATCH -o %x_%j.out            # Path to the standard output file

# Get MASTER_ADDR
nodes=$( scontrol show hostnames $SLURM_JOB_NODELIST )
nodes_array=($nodes); echo "node list: " ${nodes_array[*]}
head_node=${nodes_array[0]}; echo "HEAD_NODE=$head_node"
MASTER_ADDR=$(srun --nodes=1 --ntasks=1 -w "$head_node" bash -c "hostname -I | awk '{print \$1}'")

# Get GPU count
IFS=',' read -ra gpu_array <<< "$SLURM_JOB_GPUS"
GPUS_PER_NODE=${#gpu_array[@]}

export CUDA_VISIBLE_DEVICES=0,1 #,2,3,4,5,6,7
export OMP_NUM_THREADS=1

SIF="/work/TWCC_cntr/pytorch_23.08-py3.sif"
export SINGULARITY="singularity run --nv -B /work:/work $SIF"

export LAUNCHER="torchrun \
 --nnodes ${SLURM_JOB_NUM_NODES} \
 --nproc_per_node ${GPUS_PER_NODE} \
 --master_addr $MASTER_ADDR \
 --master_port 1234 \
"
export CMD="multigpu_torchrun.py 10 10"

srun --mpi=pmix run.sh
```

run.sh

```
#!/bin/bash
RUN="$SINGULARITY $LAUNCHER --node_rank ${SLURM_NODEID} $CMD"
echo "${SLURM_NODEID}, $RUN"; $RUN
```

Example 5 : pytorch-imagenet

```
#!/bin/bash
#SBATCH -A GOV109092           # iService Project id
#SBATCH -J resnet_1x8GPU             # job name
#SBATCH -p gp1d                # partition gtest gp1d, gpNCHC_LLM
#SBATCH --nodes=1              # Maximum number of nodes to be allocated
#SBATCH --cpus-per-task=4      # Number of cores per MPI task
#SBATCH --ntasks-per-node=1    # Number of MPI tasks (i.e. processes)
#SBATCH --gres=gpu:2
#SBATCH -o %x_%j.out           # Path to the standard output file

SIF="/work/TWCC_cntr/pytorch_23.08-py3.sif"
SINGULARITY="singularity run -B /work:/work --nv $SIF"

CMD="python main.py \
-a resnet50 \
--batch-size 256 \
--print-freq 100 \
--epochs 1 \
--rank 0 \
--world-size 1 \
--multiprocessing-distributed \
--dist-url tcp://127.0.0.1:1234 \
--dummy"
#--dummy /work/$(whoami)/dataset/imagenet
#--batch-size: 256 for 1 GPU, 512 for 2 GPU, 2048 for 8 GPU

echo "srun $SINGULARITY $CMD"
srun $SINGULARITY $CMD
```

https://github.com/pytorch/examples/tree/main/imagenet

跑單節點

sbatch run_1node.slurm

在TWCC HPC 用第二個方法 跑多節點

run_multinodes.slurm

```
#!/bin/bash
#SBATCH -A <Project_id>           # iService Project id
#SBATCH -J resnet_2x2GPU              # job name
#SBATCH -p gp1d                # partition gtest gp1d, gpNCHC_LLM
#SBATCH --nodes=2              # Maximum number of nodes to be allocated
#SBATCH --cpus-per-task=4      # Number of cores per MPI task
#SBATCH --ntasks-per-node=1    # Number of MPI tasks (i.e. processes)
#SBATCH --gres=gpu:2
#SBATCH -o %x_%j.out           # Path to the standard output file

nodes=$( scontrol show hostnames $SLURM_JOB_NODELIST )
nodes_array=($nodes); echo "node list: " ${nodes_array[*]}
head_node=${nodes_array[0]}; echo "HEAD_NODE=$head_node"
MASTER_ADDR=$(srun --nodes=1 --ntasks=1 -w "$head_node" bash -c "hostname -I | awk '{print \$1}'")

#get GPU count
IFS=',' read -ra gpu_array <<< "$SLURM_JOB_GPUS"
GPUS_PER_NODE=${#gpu_array[@]}; echo "GPUS_PER_NODE=$GPUS_PER_NODE"

export CUDA_VISIBLE_DEVICES=0,1 #,2,3,4,5,6,7
export OMP_NUM_THREADS=1

SIF="/work/TWCC_cntr/pytorch_23.08-py3.sif"
export SINGULARITY="singularity run --nv -B /work:/work $SIF"

export CMD="python main.py \
-a resnet50 \
--batch-size 128 \
--print-freq 100 \
--epochs 1 \
--world-size ${SLURM_JOB_NUM_NODES} \
--multiprocessing-distributed \
--dist-url tcp://${MASTER_ADDR}:1234 \
--dummy
"
#--dummy /home/$(whoami)/imagenet_tfrecord
# batch-size: 256 for 1 GPU, 512 for 2 GPU, 2048 for 8 GPU

srun --mpi=pmix run.sh
```

run.sh

```
#!/bin/bash
RUN="$SINGULARITY $CMD --rank ${SLURM_NODEID}"
echo "${SLURM_NODEID}, $RUN"; $RUN
```

DeepSpeed

Writing DeepSpeed Models

```
import deepspeed

parser = argparse.ArgumentParser(description='My training script.')
parser.add_argument('--local_rank', type=int, default=-1,
                    help='local rank passed from distributed launcher’)
parser.add_argument(....)
parser = deepspeed.add_config_arguments(parser) # 取得參數

deepspeed.init_distributed() # 分散式訓練初始化

# Initialize DeepSpeed for the model
model_engine, optimizer, _, _ = deepspeed.initialize(args=cmd_args,
                                                     model=model,
                                                     model_parameters=params)
# Training
for step, batch in enumerate(data_loader):
    #forward() method
    loss = model_engine(batch)

    #runs backpropagation
    model_engine.backward(loss)

    #weight update
    model_engine.step()
```

- https://github.com/deepspeedai/DeepSpeedExamples/blob/master/training/imagenet/main.py
- https://www.deepspeed.ai/getting-started/

DeepSpeed configuration, json

ds_fp16_config.json

```
{
    "train_batch_size": 256,
    "gradient_accumulation_steps": 1,
    "steps_per_print": 50,

    "optimizer": {
            "type": "Adam",
            "params": {
                "lr": 0.001,
                "betas": [
                        0.8,
                        0.999
                ],
                "eps": 1e-8,
                "weight_decay": 3e-7
            }
    },

    "zero_optimization": {
            "stage": 0
    },
    "zero_allow_untested_optimizer": true,
    "fp16": {
            "enabled": true,
            "auto_cast": true
    },
    "gradient_clipping": 0,
    "prescale_gradients": false,
    "cuda_visible_devices": 0,
    "wall_clock_breakdown" : false
}
```

ds_fp16_z1_config.json

```
{
    "train_batch_size": 256,
    "gradient_accumulation_steps": 1,
    "steps_per_print": 50,

    "optimizer": {
      "type": "Adam",
      "params": {
          "lr": 0.001,
          "betas": [
            0.8,
            0.999
          ],
          "eps": 1e-8,
          "weight_decay": 3e-7
      }
    },

    "zero_optimization": {
      "stage": 1
    },
    "zero_allow_untested_optimizer": true,
    "fp16": {
      "enabled": true,
      "auto_cast": true
    },
    "gradient_clipping": 0,
    "prescale_gradients": false,
    "cuda_visible_devices": 0,
    "wall_clock_breakdown" : false
}
```

https://www.deepspeed.ai/docs/config-json/

方法1： Launching DeepSpeed Training

國家實驗研究院 NATIONAL INSTITUTES OF APPLIED RESEARCH

hostfile

worker-1 slots=2

worker-2 slots=2

```
deepspeed --hostfile=myhostfile <client_entry.py> <client args> \
 --deepspeed --deepspeed_config ds_config.json
```

https://www.deepspeed.ai/getting-started/

方法2：torchrun (Elastic launch)

```
nodes=$( scontrol show hostnames $SLURM_JOB_NODELIST )
nodes_array=($nodes) ; echo "node list:" ${nodes_array[*]}
head_node=${nodes_array[0]}
head_node_ip=$(srun --nodes=1 --ntasks=1 -w "$head_node" hostname --ip-address)

#get GPU count
IFS=',' read -ra gpu_array <<< "$SLURM_JOB_GPUS"
GPUS_PER_NODE=${#gpu_array[@]}; echo "GPUS_PER_NODE=$GPUS_PER_NODE"

SIF="/work/$(whoami)/image/pytorch_23.01-ds.sif"
SINGULARITY="singularity run -B /work:/work --nv $SIF"

LAUNCHER="torchrun \
--nnodes $SLURM_NNODES \
--nproc_per_node ${GPUS_PER_NODE} \
--rdzv_id $RANDOM \
--rdzv_backend c10d \
--rdzv_endpoint $head_node_ip:1234 \
"
CMD="$LAUNCHER main.py \
--deepspeed \
--deepspeed_config config/ds_config.json \
…"

srun $SINGULARITY $CMD
```

製作有Deepspeed的Singularity映像檔

deepspeed.def

```
Bootstrap: docker
From: nvcr.io/nvidia/pytorch:23.01-py3
Stage: build

%post
    apt-get update && apt-get -y upgrade
    python -m pip install --upgrade pip
    python -m pip --no-cache-dir install deepspeed==0.8.0
```

Release date
pytorch:23.01-py3 , 02/01/2023
deepspeed 0.8.0, Jan. 18 2023

製作時間 10 mins

```
sudo singularity build pytorch_23.01-ds.sif deepspeed.def
```

Example 6 : DeepSpeed-imagenet

```
#!/bin/bash
#SBATCH -A <Project_id>           # iService Project id
#SBATCH -J ds_1x2GPU  # job name
#SBATCH -p gp1d                # partition gtest gp1d, gpNCHC_LLM
#SBATCH --nodes=1              # Maximum number of nodes to be allocated
#SBATCH --cpus-per-task=4      # Number of cores per MPI task
#SBATCH --ntasks-per-node=1    # Number of MPI tasks (i.e. processes)
#SBATCH --gres=gpu:2
#SBATCH -o %x_%j.out           # Path to the standard output file

SIF="/work/$(whoami)/image/pytorch_23.01-ds.sif"
SINGULARITY="singularity run -B /work:/work --nv $SIF"

CMD="deepspeed main.py \
--deepspeed --deepspeed_config config/ds_config.json \
-a resnet50 \
--batch-size 512 \
--epochs 1 \
--print-freq 100 \
--world-size 1 \
--multiprocessing_distributed \
--dummy"
#--multiprocessing_distributed \
# batch-size: 256 for 1 GPU, 512 for 2 GPU, 2048 for 8 GPU

echo "srun $SINGULARITY $CMD"
srun $SINGULARITY $CMD
```

執行一個計算節點

sbatch run_1node.slurm

https://github.com/deepspeedai/DeepSpeedExamples/tree/master/training/imagenet

用第一個方法跑多節點

run_multinodes.slurm

```
#!/bin/bash
#SBATCH -A <Project_id>          # iService Project id
#SBATCH -J ds_2x2GPU             # job name
#SBATCH -p gp1d                  # partition gtest gp1d, gpNCHC_LLM
#SBATCH --nodes=2                # Maximum number of nodes to be allocated
#SBATCH --cpus-per-task=4        # Number of cores per MPI task
#SBATCH --ntasks-per-node=1      # Number of MPI tasks (i.e. processes)
#SBATCH --gres=gpu:2
#SBATCH -o %x_%j.out             # Path to the standard output file

function makehostfile() {
perl -e '$slots=split /,/, $ENV{"SLURM_STEP_GPUS"};
$slots=2 if $slots==0; # workaround 8 gpu machines
@nodes = split /\n/, qx[scontrol show hostnames $ENV{"SLURM_JOB_NODELIST"}];
print map { "$b$_ slots=$slots\n" } @nodes'
}
makehostfile > hostfile

export OMP_NUM_THREADS=1
rm hostfile *.pth.tar

SIF="/work/$(whoami)/image/pytorch_23.01-ds.sif"
SINGULARITY="singularity run -B /work:/work --nv $SIF"

LAUNCHER="deepspeed --hostfile=hostfile"
```

```
CMD="$LAUNCHER main.py \
--deepspeed \
--deepspeed_config config/ds_fp16_z1_config.json \
-a resnet50 \
--batch-size 512 \
--epochs 1 \
--print-freq 100 \
--world-size 2 \
--multiprocessing_distributed \
--dummy"

RUN="srun $SINGULARITY $CMD"
echo $RUN; $RUN
```

用第二個方法跑多節點 torchrun

國家實驗研究院 NATIONAL INSTITUTES OF APPLIED RESEARCH

```
#!/bin/bash
#SBATCH -A <Project_id>           # iService Project id
#SBATCH -J ds_2x8GPU             # job name
#SBATCH -p gp1d                  # partition gtest gp1d, gpNCHC_LLM
#SBATCH --nodes=2                # Maximum number of nodes to be allocated
#SBATCH --cpus-per-task=4        # Number of cores per MPI task
#SBATCH --ntasks-per-node=1      # Number of MPI tasks (i.e. processes)
#SBATCH --gres=gpu:2
#SBATCH -o %x_%j.out             # Path to the standard output file

nodes=$( scontrol show hostnames $SLURM_JOB_NODELIST )
nodes_array=($nodes) ; echo "node list:" ${nodes_array[*]}
head_node=${nodes_array[0]}
head_node_ip=$(srun --nodes=1 --ntasks=1 -w "$head_node" hostname --ip-address)

#get GPU count
IFS=',' read -ra gpu_array <<< "$SLURM_JOB_GPUS"
GPUS_PER_NODE=${#gpu_array[@]}; echo "GPUS_PER_NODE=$GPUS_PER_NODE"

export OMP_NUM_THREADS=1

SIF="/work/$(whoami)/image/pytorch_23.01-ds.sif"
SINGULARITY="singularity run -B /work:/work --nv $SIF"

LAUNCHER="torchrun \
--nnodes $SLURM_NNODES \
--nproc_per_node ${GPUS_PER_NODE} \
--rdzv_id $RANDOM \
--rdzv_backend c10d \
--rdzv_endpoint $head_node_ip:1234 \
"
```

```
CMD="$LAUNCHER main.py \
--deepspeed \
--deepspeed_config config/ds_config.json \
-a resnet50 \
--batch-size 512 \
--epochs 1 \
--print-freq 100 \
--world-size 2 \
--multiprocessing_distributed \
--dummy"

RUN="srun $SINGULARITY $CMD"
echo "$RUN"; $RUN
```

run_multinodes1.slurm


thank you
danke
謝謝
ngiyabonga
teşekkür ederim
gracias
merci
obrigado
bedankt
dziękuję
sukriya
kop khun krap
go raibh maith agat
mochchakkeram
terima kasih
감사합니다
ありがとう
arigatô
dakujem
grazie
спасибо
Баярлалаа
рахмат
dank je
tapadh leat
mauruuru
hvala
sagolun
takk
taiku
мерси
ευχαριστώ
enkosi
nanni
nandri
kiitos
dankie
dhanyavad
köszönöm
faafetai lava
vinaka
спасибі
blagodaram
mersi
kia ora
barka
welalin
tack
spas
misaotra
matondo
paldies
grazzi
mahalo
хвала
asante
manana
obrigada
tenki
murakoze
djiere dieuf
tau
дякую
mamnun
chokrane
chnorakaloutioun
gratias ago
gràcies
sulpáy
akun
dankon
aciu
bayarlalaa
gracie
sobodi
dėkuji
mèsi
didi madloba
najis tuke
kam sah hamnida
rahmat
tanemirt
rahmet
xiexie
diolch
dhanyavadagalu
shukriya
mercé
trugarez